

# Asynchronous, Generation-Free ABC: Streaming AMIS Reweighting for Heterogeneous Simulator Workloads on HPC

First Author<sup>1\*</sup> and Second Author<sup>1</sup>

<sup>1\*</sup>Department, Organization, City, Country.

\*Corresponding author(s). E-mail(s): [first.author@example.com](mailto:first.author@example.com);  
Contributing authors: [second.author@example.com](mailto:second.author@example.com);

## Abstract

Sequential approximate Bayesian computation advances in generations, and a generation cannot end before its slowest simulation. On a parallel machine this barrier idles every worker that finishes early. For identically distributed simulation times its cost is the straggler factor of the workload: the expected maximum of  $W$  times over their mean. We introduce a generation-free ABC sampler that updates its proposal after every arriving evaluation, weights every evaluated particle by a streaming form of adaptive multiple importance sampling, and estimates the posterior from the full evaluated history. We prove the estimator consistent up to a fidelity ratio, find it calibrated where a kernel-matched pyABC baseline under-covers, and measure four things. (i) The barrier’s cost, predicted from the asynchronous run’s timing, matches a barrierized twin of our sampler over measured costs from  $1.2\times$  to  $400\times$ : within  $1\%$  under a persistent straggler,  $20\%$  under lognormal heterogeneity, and  $10\%$  on a Cellular Potts tissue simulator. (ii) Removing the barrier yields a tighter tolerance. On the tissue simulator the asynchronous sampler completes  $2.1\text{--}2.4\times$  the simulations of pyABC ( $2.0\times$  attributable to the barrier), is  $1.6\times$  more efficient per simulation, and ends  $4.1\times$  tighter at equal wall clock. (iii) The throughput advantage turns on at a few milliseconds per simulation. (iv) Once the starting bandwidth is set from the prior-predictive discrepancy scale, the method recovers a two-parameter tissue posterior tighter than pyABC’s and on a larger effective sample; that effective sample size, a few multiples of the archive size, is the remaining limit.

**Keywords:** approximate Bayesian computation, sequential Monte Carlo, adaptive multiple importance sampling, asynchronous algorithms, high-performance computing, likelihood-free inference

# 1 Introduction

Approximate Bayesian computation (ABC) is the standard route to Bayesian inference when a simulator can be run but its likelihood cannot be evaluated [1]. Its sequential forms, ABC-SMC and ABC-PMC [2–5], move a population of particles through a sequence of shrinking tolerances. Every implementation we know of does so in generations: the proposal and the tolerance are updated once the whole population has been simulated. That is a synchronization barrier. On a parallel machine with  $W$  workers a generation ends when its slowest simulation does, and every worker that finished earlier waits. If simulation times  $T_i$  are drawn from a distribution with mean  $\mu$ , a generation of  $W$  takes about  $\mathbb{E}[\max_{i \leq W} T_i]$ . The barrier therefore costs a factor  $\mathbb{E}[\max_{i \leq W} T_i]/\mu$ , the *straggler factor* of the workload, which is 1 for a homogeneous simulator and grows with  $W$  and with the spread of the runtime distribution. Simulators worth running on such machines are rarely homogeneous: the cost of a cell-based tissue simulation tracks the number of cells it produces, which varies by orders of magnitude across a prior.

This paper removes the generation. We propose an ABC sampler that rebuilds its proposal mixture after every arriving evaluation, from a sliding archive of the  $k$  best particles evaluated so far. Every evaluated particle is weighted against the mixture of proposals actually used; this weighting is the streaming limit of adaptive multiple importance sampling (AMIS) [6]. The estimator the method reports is a pure function of the evaluated history. Nothing has to be synchronized between workers, so the sampler runs on the barrier-free island model of the Propulate optimization engine [7]. Look-ahead scheduling [8] attacks the same idle time by starting the next generation early on free workers. It keeps the generation and recovers the 10–50% that the boundary wastes. We remove the boundary and recover the factor. The practical comparison is with pyABC [9] run with the same kernel and bandwidth schedule: on a tissue simulator our sampler completes  $2.4\times$  its simulations in the same hour, ends at a  $4.1\times$  tighter tolerance and recovers a tighter posterior on a larger effective sample, and under a persistent straggler pyABC’s throughput falls more than a hundredfold and ours by less than half.

Our contributions are the following; C1–C4 label the four claims that §6 tests, one per subsection.

1. A generation-free, single-arrival-driven ABC sampler combining smooth-kernel ABC with streaming AMIS reweighting, whose posterior estimate is a replayable function of the evaluated history (§3).
2. A barrierized twin of that sampler, and the measurement that the barrier’s cost is predicted from the asynchronous run’s timing alone, across two and a half orders of magnitude of that cost (C1, §6.1).
3. The measurement that the gain converts into a tighter tolerance against a kernel-matched pyABC baseline on a tissue simulator, and that the advantage turns on at a few milliseconds per simulation (C2, §6.2; C3, §6.3).
4. Consistency of the estimator up to a fidelity ratio that we decompose and partly measure (§4; proofs and a central limit theorem for damped-adaptation variants in the appendix); calibration checks in one and four dimensions and under multimodality, recovery of a two-parameter Cellular Potts posterior, and the two settings

that bound what the estimator delivers: the starting bandwidth and the archive size (C4, §6.4).

## 2 Background and Related Work

ABC replaces the likelihood with simulation and a discrepancy: for observed data  $y$ , parameter  $\theta$  and a discrepancy  $\rho$  between simulated and observed summaries, the ABC target at bandwidth  $\epsilon$  is

$$\pi_\epsilon(\theta \mid y) \propto \pi(\theta) L_\epsilon(\theta), \quad L_\epsilon(\theta) = \mathbb{E}_{\rho \sim p(\rho \mid \theta)}[K_\epsilon(\rho)], \quad (1)$$

with  $\pi$  the prior and  $K_\epsilon$  the ABC kernel. The classical kernel is the hard indicator  $\mathbf{1}[\rho < \epsilon]$ ; a smooth kernel such as the Gaussian  $\exp(-\rho^2/2\epsilon^2)$  [10, 11] removes the discontinuity in the likelihood approximation. The generation-staged family (ABC-SMC [2, 3], ABC-PMC [4], and adaptive variants [5, 12, 13]) advances through intermediate targets with a mixture proposal built from the previous stage’s accepted particles. It weights each stage against that single previous proposal, so the full population is needed before the next proposal can be formed. Adaptive multiple importance sampling (AMIS) [6] instead re-weights *all* past particles against the cumulative mixture of the proposals used (the balance heuristic of [14]). It belongs to the adaptive-importance-sampling line of population Monte Carlo [15], surveyed in [16], and is consistent under regularity conditions [17]. Our method takes its streaming limit: stage size one, single-arrival-driven adaptation.

Parallel and off-barrier SMC has been studied for state-space models: the island particle model [18], asynchronous anytime particle methods [19], and parallel resampling [20]. None of these targets ABC, and none combines barrier-free execution with AMIS-style cumulative-mixture reweighting. Distributed ABC frameworks such as pyABC [9, 21], ABCpy [22], and ELFI [23] parallelize *simulation* effectively within a generation.

## 3 Method

### 3.1 Design constraints and history-based state

We design the algorithm so that the estimator is a pure function of the evaluated history: the archive, the reconstructed bandwidth and the posterior weights are all recomputed from the recorded log. A crashed run is therefore recovered by replaying that log rather than by checkpointing sampler state, and the estimator is identical whether it is formed online or afterwards.

The running sampler does carry an online snapshot buffer and two scheduler throttles between calls, so a restart reproduces the estimator over whatever history was evaluated, not the trajectory the run would have taken (Appendix D). The Propagate propagator interface [7], which hands the whole evaluated history to each call and takes one candidate back, matches this contract. Define the evaluated history at call  $n$  as

$$\mathcal{H}_n = \{(\theta_i, \rho_i, \epsilon_i, \tau_i, w_i)\}_{i=1}^n, \quad (2)$$

where  $\theta_i$  is the parameter,  $\rho_i$  the discrepancy,  $\epsilon_i$  the bandwidth active when  $\theta_i$  was proposed,  $\tau_i$  the proposal-time index, and  $w_i$  the stored core importance weight  $\pi/q_{\tau_i}$ .

### 3.2 Tolerance reconstruction and monotonicity

Each candidate proposed from the archive stores the bandwidth active at proposal time; the uniform bootstrap draws that precede the first archive (§3.5) store none. The effective bandwidth reconstructed from history is the running minimum of the stored bandwidths,  $\epsilon_{\text{hist}} = \min_i \epsilon_i$  over the stamped particles (falling back to the initial tolerance while none is stamped). The scheduler proposes  $\epsilon_{\text{sched}}$ , and the bandwidth used is  $\epsilon_n = \min(\epsilon_{\text{hist}}, \epsilon_{\text{sched}})$ , which makes the bandwidth non-increasing without any mutable propagator state. The schedule starts from  $\epsilon_0$  (`tol_init`) and tightens at a bounded rate toward the bandwidth at which  $2k$  evaluated particles lie within it: each rank’s scheduler searches once per  $k$  of its own calls, and a search may at most halve the bandwidth. On an expensive simulator, or on any run in which no rank makes many multiples of  $k$  calls, that transient outlasts the run, so  $\epsilon_0$  is set from the prior-predictive discrepancy scale (§6.4).

### 3.3 Archive and smooth-kernel proposal

The reconstructed archive is the top- $k$  history members by lowest discrepancy,  $A_n = \text{Top}_k(\{\theta_i\}, \text{order by } \rho_i)$ . The proposal mixture uses kernel-weighted archive members,

$$q_n(\theta) = \sum_{j \in A_n} \tilde{W}_j^{(n)} K_\Sigma(\theta - \theta_j), \quad \tilde{W}_j^{(n)} \propto w_j \cdot K_{\epsilon_n}(\rho_j), \quad (3)$$

with  $\sum_j \tilde{W}_j^{(n)} = 1$ . The perturbation kernel  $K_\Sigma$  is Gaussian with covariance  $\Sigma_n = s \widehat{\text{Cov}}_{\tilde{W}}(A_n)$  estimated from the kernel-weighted archive ( $s$  is the `perturbation_scale` hyperparameter), factored once via Cholesky. Smooth kernels remove the prior-vs-archive discontinuity present in hard-threshold ABC-PMC: every archive member contributes continuously through its kernel weight.

### 3.4 Streaming AMIS importance weight

A particle  $\theta^*$  drawn from  $q_n$  receives an importance weight under the balance heuristic over a sliding ring buffer  $\mathcal{S}$  of past proposals (size  $S$ , sampled every `amis_interval` calls):

$$w^* = \frac{\pi(\theta^*)}{\bar{q}_n^{\text{on}}(\theta^*)}, \quad \bar{q}_n^{\text{on}}(\theta) = \frac{1}{1 + |\mathcal{S}|} \left[ q_n(\theta) + \sum_{s \in \mathcal{S}} q_s(\theta) \right]. \quad (4)$$

$|\mathcal{S}| = 0$  recovers single-proposal weighting; we use  $S = 20$ , at a modest per-call cost, because  $S = 0$  leaves the estimator over-confident in one and two dimensions in the calibration sweep of Appendix F (mean coverage deviation  $-0.088$ ), and every  $S \geq 5$  removes that deviation. A particle’s weight, assigned at proposal time, is corrected by the cumulative-mixture denominator at every subsequent use (the retroactive AMIS step). Each snapshot  $q_s$  is the proposal reconstructed from the history truncated at call  $s$ ; the estimator rebuilds its own snapshots from  $\mathcal{H}_n$  (Appendix C).

### Three weights, one posterior.

The method uses three weightings. The *adaptation weights*  $\tilde{W}_j^{(n)}$  of (3) are kernel weights over the archive and define the next proposal. The *proposal-time weight*  $w^*$  of (4) is stored with each particle and drives parent selection and the online effective-sample-size diagnostic, and nothing else. The *posterior estimate* is the retroactive estimator  $\hat{\pi}_n$  of (7), recomputed from the history on demand: every evaluated particle is reweighted by  $\pi(\theta_i)K_{\epsilon_n}(\rho_i)/\bar{q}_n(\theta_i)$ , where  $\bar{q}_n$  is the  $m$ -snapshot denominator with a defensive prior component specified in §4. That denominator is a fixed-size stand-in for the mixture the sample was drawn from. Computing  $\hat{\pi}_n$  is a one-time  $\mathcal{O}(nk)$  pass over the history, run after the timed budget and excluded from every throughput measurement (Appendix C).

## 3.5 Update loop

Algorithm 1 assembles the pieces above into the single-arrival update the propagator runs on each call. Every quantity it uses is reconstructed from the history  $\mathcal{H}_n$  passed in; until the history holds at least  $k$  members it emits uniform prior draws to seed the archive. Fig. 1 draws one such update beside one generation of ABC-PMC, step by step.

---

### Algorithm 1 History-reconstructed, single-arrival-driven ABC update

---

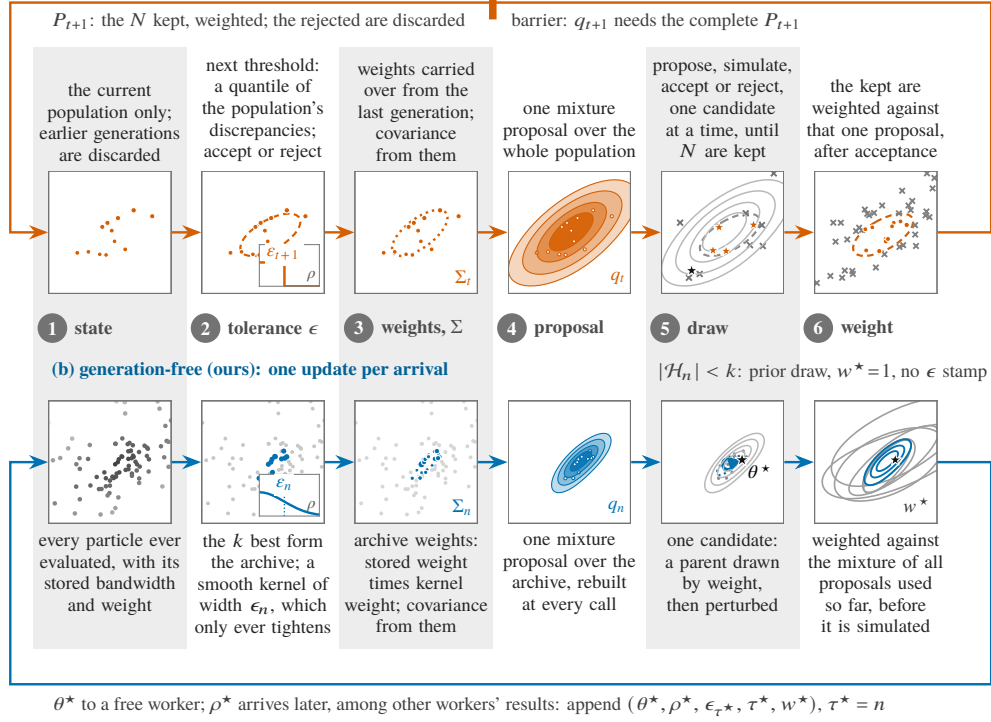
**Require:** History  $\mathcal{H}_n$ , archive size  $k$ , initial bandwidth  $\epsilon_0$ , snapshot buffer  $\mathcal{S}$

- 1: Reconstruct  $\epsilon_{\text{hist}}$  from stored tolerances; compute  $\epsilon_{\text{sched}}$ ; set  $\epsilon_n \leftarrow \min(\epsilon_{\text{hist}}, \epsilon_{\text{sched}})$
  - 2: **if**  $|\mathcal{H}_n| < k$  **then**
  - 3:   emit a uniform prior draw (bootstrap) ▷ covers  $\bar{q}$  on the whole box
  - 4: **else**
  - 5:   select archive  $A_n$  (top- $k$  by  $\rho$ ); form  $\tilde{W}^{(n)}$  in log-space via  $K_{\epsilon_n}$
  - 6:   build  $\Sigma_n$ , Cholesky-factor once; sample  $\theta^*$  by perturbing a  $\tilde{W}^{(n)}$ -weighted parent
  - 7:   compute  $w^*$  via the AMIS denominator (4); periodically snapshot  $q_n$  into  $\mathcal{S}$
  - 8: **end if**
  - 9: **return**  $\theta^*$  for external simulation and discrepancy evaluation
- 

## 4 Theoretical Analysis

We prove consistency, up to a fidelity ratio, for the estimator as implemented, computed by a post-hoc pass over the log the algorithm writes. The proof is self-contained rather than a corollary of AMIS theory, because the published consistency results for recycling schemes [17] restrict the adaptation so that the weighting stage decouples, which a single-arrival scheme does not do. The proof also carries the implementation’s departures from the ideal inside the statement, as a single ratio  $r$ . A central limit theorem for damped-adaptation variants, with a per-run variance estimator, and the decomposition and partial measurement of  $r$  are in Appendix A; the proofs are in Appendix B.

(a) textbook hard-threshold ABC-PMC: one update per generation



**Fig. 1** One generation of ABC-PMC (a) against one single-arrival update of Algorithm 1 (b), step by step on a two-dimensional toy ( $k = N = 12$ ). Row (a) is textbook PMC with a quantile schedule and a hard threshold,  $\omega_i^{(t+1)} \propto \pi(\theta_i)/q_t(\theta_i)$  normalized over the  $N$  kept; it is not the matched-kernel pyABC baseline of §5.2. In row (b),  $\epsilon_n = \min(\epsilon_{\text{hist}}, \epsilon_{\text{sched}})$ ,  $\Sigma_n = L_n L_n^\top$ , and  $w^* = \pi(\theta^*)/\bar{q}_n^{\text{on}}(\theta^*)$  with the denominator of (4) over the stored proposals.  $w^*$  is assigned before the simulation and stored with the particle; the posterior estimate (7) reweights every particle of  $\mathcal{H}_n$  afresh. Bootstrap draws ( $|\mathcal{H}_n| < k$ ) are prior draws with  $w^* = 1$  and no bandwidth stamp. Point sizes vary where weights do (a1–a3 by  $\omega^{(t)}$ , a6 by  $\omega^{(t+1)}$ , b3 by  $\bar{W}_j$ ); shading in b1 is  $\rho_i$  (light: far); dotted ellipses are  $\Sigma_t$  and  $\Sigma_n$ ; gray contours are the current proposal in a5 and b5 and the stored proposals  $q_s$  in b6; the dashed ellipse in row (a) is the hard acceptance region; the inset in column 2 is the kernel over  $\rho$ , the Gaussian  $K_{\epsilon_n}$  in (b) against the indicator in (a).

The analysis is organized around a single question, because that is all the proofs turn out to need: *is the denominator the estimator divides by the density its sample was actually drawn from?* Write  $\tilde{q}_i$  for the conditional density of the  $i$ -th candidate given the past, in the sense of the filtration fixed by Assumption 4. It absorbs every internal departure of the implementation (the top- $k$  archive rule, the capped in-box rejection, the underflow redraws, the bootstrap phase), and it is the *only* way the sampler enters Theorem 1. Write

$$\bar{q}_n^* := \frac{1}{n} \sum_{i \leq n} \tilde{q}_i \quad (5)$$

for the mixture the pooled sample was drawn from, against the  $m$ -snapshot denominator

$$\bar{q}_n = w_n \pi + (1 - w_n) \sum_{s=1}^m \alpha_{s,n} q_{\tau_s}, \quad w_n = \max(\nu_n, \delta), \quad \alpha_{s,n} \geq 0, \quad \sum_s \alpha_{s,n} = 1, \quad (6)$$

the draw-proportional deterministic mixture [14, 24] of  $m \leq S+1 = 21$  history-reconstructed proposals with a prior component whose mass is the observed bootstrap share  $\nu_n$  floored at  $\delta = 0.5/(m+1)$  (Appendix C). Throughout,  $\epsilon_\infty$  denotes the limit of the bandwidth schedule  $\{\epsilon_n\}$ , whose existence and positivity are part of Assumption 5(i);  $L_\epsilon(\theta) = \mathbb{E}[K_\epsilon(\rho) \mid \theta]$  is the smooth ABC likelihood (1),  $L_\epsilon^{(2)}(\theta) = \mathbb{E}[K_\epsilon(\rho)^2 \mid \theta]$ , and the estimator is

$$\hat{\pi}_n(f) = \frac{\sum_{i=1}^n W_{i,n} f(\theta_i)}{\sum_{i=1}^n W_{i,n}}, \quad W_{i,n} = \frac{\pi(\theta_i) K_{\epsilon_n}(\rho_i)}{\bar{q}_n(\theta_i)}. \quad (7)$$

### Assumptions.

**Assumption 1** (kernel).  $K_\epsilon : [0, \infty) \rightarrow [0, 1]$  is measurable with  $K_\epsilon(0) = 1$  (unit peak, not unit integral, as is standard for a probabilistic-acceptance ABC kernel), and for every  $a \in (0, \epsilon_0]$  there is  $L_K(a) < \infty$  with  $|K_\epsilon(\rho) - K_{\epsilon'}(\rho)| \leq L_K(a)|\epsilon - \epsilon'|$  for all  $\rho \geq 0$  and  $\epsilon, \epsilon' \in [a, \epsilon_0]$ . The limiting target is non-degenerate:  $Z_{\epsilon_\infty} = \int_\Theta \pi L_{\epsilon_\infty} > 0$ .

**Assumption 2** (denominator class).  $\Theta \subset \mathbb{R}^d$  is a non-degenerate axis-aligned box and the prior density satisfies  $0 < \pi_- \leq \pi \leq \pi_+ < \infty$  on it. Each  $q_{\tau_s}$  in (6) is a positive function on  $\Theta$  of truncated-Gaussian-mixture form: at most  $k$  components, means in  $\Theta$ , weights in the simplex, covariance  $\Sigma$  with  $\sigma_-^2 I \preceq \Sigma \preceq \sigma_+^2 I$  for fixed  $0 < \sigma_- \leq \sigma_+ < \infty$ , and per-component normalizing constants evaluated as the implementation evaluates them.

**Assumption 3** (prior floor in the denominator).  $\delta \in (0, 1/2]$  is fixed and  $w_n \geq \delta$  in (6).

**Assumption 4** (adapted filtration). There is a filtration  $(\mathcal{F}_i)$  with  $\xi_i = (\theta_i, \rho_i)$   $\mathcal{F}_i$ -measurable,  $\theta_i \mid \mathcal{F}_{i-1} \sim \tilde{q}_i$  and  $\rho_i \mid \mathcal{F}_{i-1} \vee \sigma(\theta_i) \sim p(\cdot \mid \theta_i)$ . Under sequential execution this holds by construction; under asynchronous execution it is an assumption (Appendix B).

**Assumption 5** (fidelity and stabilization). (*i, consistency*)  $\epsilon_n \downarrow \epsilon_\infty > 0$  almost surely ( $\epsilon_\infty$  may be random), and the denominator tracks the draw mixture up to a limiting ratio: for some  $r$  with  $0 < r_- \leq r \leq r_+ < \infty$ ,

$$\|\bar{q}_n^* / \bar{q}_n - r\|_\infty \longrightarrow 0 \quad \text{a.s.} \quad (8)$$

(*ii, central limit theorem*) A stabilization rate and deterministic limits, needed only for Theorem 3 and stated in Appendix A.

Assumption 1 holds for the Gaussian kernel with  $L_K(a) = 2/(ea)$  and Epanechnikov with  $2/a$ ; it excludes the hard indicator, which is not Lipschitz in  $\epsilon$ . This is one concrete reason the method uses smooth kernels. Assumptions 2 and 3 constrain only the  $m$ -snapshot denominator  $\bar{q}_n$ , which the implementation builds to that exact recipe. Assumption 5 carries everything else. For consistency, (8) is the whole of the sampler’s contribution: the sampler need not take any particular form, but whatever it does must be asymptotically reflected in the denominator, up to a ratio  $r$  that the results carry. The central limit theorem needs in addition the per-draw stabilization rate and the deterministic limits of clause (ii).

**Theorem 1** (consistency). *Let Assumptions 1–4 and 5(i) hold. Then for every bounded measurable  $f : \Theta \rightarrow \mathbb{R}$ ,*

$$\hat{\pi}_n(f) \longrightarrow \pi_{\epsilon_\infty}^r(f) := \frac{\int_{\Theta} f \pi L_{\epsilon_\infty} r}{\int_{\Theta} \pi L_{\epsilon_\infty} r} \quad \text{almost surely.} \quad (9)$$

**Corollary 2** (exact fidelity). *If  $r \equiv 1$ , that is, if the denominator is asymptotically the density the sample was drawn from, then  $\hat{\pi}_n(f) \rightarrow \pi_{\epsilon_\infty}(f)$ , the smooth-ABC posterior at the limiting bandwidth.*

**What  $r$  is, and how much of it is measured.**

Everything specific to the code enters through  $r$ : the prior floor in the denominator, the  $m$ -point snapshot quadrature standing in for the  $n$ -point mixture, the product-of-marginals truncation constant, and the capped in-box rejection and underflow redrews. The first two can be estimated from a stored run by recomputing the denominator at large  $m$  and at the observed prior share. On the five Cellular Potts production runs the posterior estimate lies 0.2–0.5% in total variation from that reference; on a one-dimensional Gaussian history, 0.26%. The other two are bounded, not measured, so this is a partial accounting of  $r$  and the results are conditional on its finite positive limit (Appendix A, Table B1).

## 5 Experimental Design

### 5.1 Benchmarks

Table 1 lists the five configurations. The two cheap benchmarks, a one-dimensional Gaussian mean with an analytic posterior and the four-dimensional g-and-k distribution [25] with a reference posterior, are where calibration and recovery can be scored against a reference and where the cost of coordinating a near-instant simulator is exposed. The Gaussian mean also hosts the two studies with *injected* runtime heterogeneity, in which the runtime distribution is known by construction: a persistently slow worker and a per-evaluation lognormal runtime multiplier. Lotka–Volterra is a stochastic simulator whose cost varies with the trajectory and whose parameters send 98% of simulations to extinction (Appendix D); it serves the scaling and boundary measurements and is not scored for posterior quality. The Cellular Potts benchmark runs the real `cellsInSilico` tissue simulator [26], a three-dimensional Cellular Potts

**Table 1** Benchmark suite. Cost is the mean wall-clock time of one simulation in the asynchronous runs (an evaluation on Cellular Potts is four replicate simulations); runtime spread is the in-run coefficient of variation of that time, or how it is injected. Budgets are per replicate; five replicates throughout except three at  $80^3$ .

| Benchmark             | $d$ | cost/sim | runtime spread                                                | budget, workers                       | role                                                                                          |
|-----------------------|-----|----------|---------------------------------------------------------------|---------------------------------------|-----------------------------------------------------------------------------------------------|
| Gaussian mean         | 1   | 0.1 ms   | injected: straggler 0–20 $\times$ ; lognormal $\sigma \leq 2$ | 300 s, 48 (16 in the straggler study) | calibration (SBC); barrier cost under a known runtime law; the cheap end of the cost boundary |
| g-and-k               | 4   | 2 ms     | none                                                          | 600 s, 48                             | 4-D calibration and recovery against a reference posterior                                    |
| Lotka–Volterra        | 4   | 4 ms     | trajectory-dependent; 98% extinct                             | 600 s, 48; scaling to 288             | strong scaling; the cost boundary                                                             |
| Cellular Potts $50^3$ | 2   | 13 s     | CV 0.05                                                       | 3600 s, 48; scaling to 384            | posterior recovery against a known truth; barrier cost on a real simulator                    |
| Cellular Potts $80^3$ | 2   | 170 s    | CV 0.13–0.23                                                  | 3600 s, 48                            | barrier cost at realistic cost and heterogeneity                                              |

model [27], end to end. Its inference problem was designed on a screening experiment (Appendix E). It has two parameters, the cell division rate and the target cell volume, log-uniform on  $[0.001, 0.2]$  and  $[200, 1200]$ . They move the two directions the summary statistics resolve (the cell count, and the cluster radius at fixed count), and the discrepancy over those two statistics is averaged over four replicate simulations per evaluation. We run it at two sizes (Table 1); the runtime spread rises with the box because runtime tracks cell count (correlation 0.99) and cell count is no longer bounded at  $80^3$ . The  $50^3$  twin and scaling experiments ran an earlier two-parameter configuration of this benchmark (division rate and motility, 5.1 s per single-simulation evaluation; Appendix E).

The Cellular Potts benchmark is a calibration instrument: its simulations take seconds to minutes rather than the hours of a production run, and its data are synthetic with a known truth.

## 5.2 Baselines and instruments

Three comparators appear, each for a different question, and each keeps one name throughout. *pyABC* is pyABC [9, 21] with its acceptor replaced by a probabilistic-rejection acceptor using the same smooth kernel and the same bandwidth schedule as the asynchronous sampler (Appendix C); it is the method a practitioner would run, and the comparator for C2 and C3. Its population of 100 on 47 simulating ranks (rank 0 dispatches) forces at least three rounds per generation, the last one 13% full, so it idles by construction even on a homogeneous simulator. The scaling study gives it a population equal to the worker count; the production comparisons do not. The *twin* is our own propagator with a collective barrier inserted before each proposal and nothing else changed. Histories are not truncated to generation boundaries, so each rank proposes from the history it has received at the barrier. The twin thus isolates the timing of synchronization from every other difference, and it is the comparator for

C1. A collective barrier needs identical call counts on every rank, so the twin runs to a fixed simulation count, the asynchronous sampler runs to a fixed wall clock, and the two are compared as rates. For the posterior comparison the asynchronous sampler is also re-run at exactly the twin’s simulation count and seeds. *Rejection ABC* spends the same simulation budget, keeps the  $k$  best draws, and is given the whole machine (all 48 ranks) rather than one rank per replicate. It is the reference for what an adaptive proposal buys per simulation. All three run over MPI on JUWELS (Appendix D).

### 5.3 Metrics

#### *Tolerance at a matched budget.*

To compare samplers on one footing we use the  $k$ -th order statistic of a method’s own discrepancies after  $n$  simulations,  $\epsilon_{(k)}(n)$ : the tolerance at which exactly  $k$  of its draws would be accepted. It needs no reference posterior, it does not depend on the bandwidth a schedule happened to report at, and it is what rejection ABC reports at by construction. Read at equal  $n$  it measures per-simulation efficiency; read at the end of equal wall-clock budgets, the net effect. Its slope in  $\log n$  says whether more simulations keep converting into a tighter tolerance. We use  $k = 100$  throughout, over completed simulations only, ordered by completion time (Appendix D).

#### *Throughput, utilization, and the predicted barrier cost.*

Throughput is simulations completed per second of wall clock; utilization is the fraction of worker time spent inside the simulator. The prediction that C1 tests is built from the asynchronous run only. Let  $D_i$  be the per-evaluation durations on that run (recorded simulation time plus the measured per-evaluation coordination overhead). A barrierized generation of  $W$  evaluations takes  $\mathbb{E}[\max_{i \leq W} D_i]$ , so the synchronous throughput is predicted as  $T_{\text{sync}} = W/\mathbb{E}[\max_{i \leq W} D_i]$ . The expectation is taken by resampling  $W$  durations from the asynchronous record, or from the injected law where the heterogeneity is injected; a persistently slow worker contributes its own mean. The predicted cost is  $T_{\text{async}}/T_{\text{sync}}$  with  $T_{\text{async}}$  the measured asynchronous throughput, and it is compared with the ratio measured against the twin. For identically distributed, uncensored durations  $T_{\text{async}} = W/\mathbb{E}[D]$  and the ratio is the straggler factor of §1. With a persistently slow worker, or when a wall-clock budget censors the long durations, the two differ, and we use the general form.

#### *Posterior quality.*

Calibration is checked by simulation-based calibration (SBC) [28]: empirical coverage of the posterior’s central intervals over 1000 trials at the 0.50/0.80/0.90/0.95 levels. Where a reference posterior exists we report the Wasserstein distance  $W_1$  between each method’s own posterior and that reference (in more than one dimension the sliced  $W_1$  over random projections [29], on the parameters’ natural scale). The reference is the analytic posterior for the Gaussian mean; for g-and-k it is the posterior given the same octile summaries the discrepancy is built from, obtained from their asymptotic law (Appendix D). The asynchronous estimator is replayed over the prefix of the history completed by each checkpoint. pyABC is scored on its last completed generation,

with pyABC’s population weights where the history was kept and uniform weights otherwise, and rejection ABC on its  $k$  best draws. On Cellular Potts, whose truth is known but whose posterior is not, we report each parameter’s *contraction*  $1 - \text{sd}/\text{sd}_{\text{prior}}$  in prior-normalized units, whether the central 90% interval covers the truth, and the effective sample size of the posterior weights.

## 6 Results

Unless noted, results are medians over five replicates with the replicate range shown; *asynchronous* is the proposed sampler, *pyABC* the kernel-matched pyABC baseline, and *twin* the barrierized copy of our own propagator (§5.2).

### 6.1 The barrier costs the straggler factor, and the cost is predictable (C1)

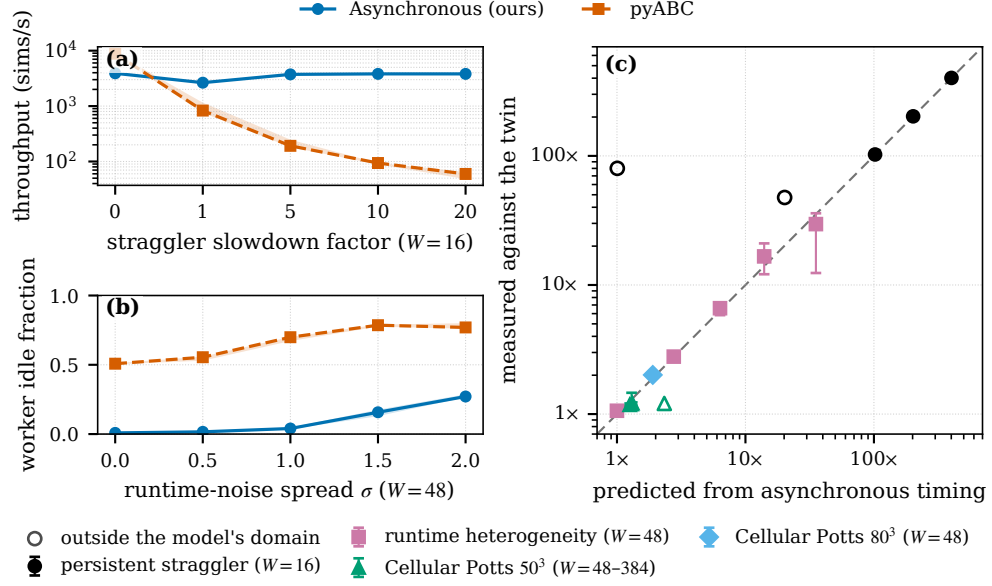
Figure 2 shows the barrier from both sides: against pyABC, which pays it (panels a and b), and against our own twin, which isolates it (panel c). Panel (c) is the paper’s central measurement. It covers every configuration on which we ran the barrierized twin: a persistently slow worker at five slowdowns, lognormal runtime heterogeneity at five spreads, and the Cellular Potts simulator at four worker counts and two sizes. For each it plots the barrier’s cost as *predicted* from the asynchronous run’s timing alone (§5.3) against the cost *measured* against the twin. Twelve configurations lie inside the model’s stated domain; over them the predicted-to-measured ratio runs from 0.84 to 1.19 with median 0.996, across measured costs from  $1.2\times$  to  $400\times$ . Three configurations lie outside the domain and are drawn open: the cost-free simulator at  $0\times$  and  $1\times$ , where the collective’s own latency dominates and the model has no latency term; and 384 Cellular Potts ranks, where the asynchronous timing sample carries a contention tail that the twin’s generations did not show.

#### *A persistent straggler.*

One of 16 workers is given a permanent post-evaluation delay of 0.1s scaled by 0, 1, 5, 10, 20 (Table 2; Fig. 2a). The prediction is a one-line calculation. The slow worker’s recorded evaluation time (2.000s at  $20\times$ ) plus the 3.6ms per-evaluation overhead measured on the fast workers is the generation time, so the twin cannot exceed  $16/2.004 = 7.98$  simulations per second. The measured asynchronous rate over that gives  $401\times$ , against a measured  $402\times$ ; at  $5\times$  and  $10\times$  the agreement is the same. At  $0\times$  and  $1\times$  the prediction fails. With no straggler to wait for, the collective itself costs about 0.3s per barrier on a 4ms workload, a term the model does not contain; coarsening the barrier to one per 112 evaluations (pyABC’s population) amortizes it. From  $5\times$  up the two barrier granularities agree with each other and with the prediction within 0.5% (0.2% at  $10\times$ , 0.06% at  $20\times$ ): once a straggler binds, the barrier’s cost does not depend on how often it is taken.

#### *Runtime heterogeneity.*

Every evaluation on 48 workers is multiplied by a lognormal factor of scale  $\sigma$ , so the slow draw is a different worker each time (Table F4, Appendix F). The control costs



**Fig. 2** The barrier against pyABC and against our own twin. (a) Throughput under a persistent straggler: one of 16 workers carries a permanent post-evaluation delay of 0.1s scaled by the factor (Gaussian mean, 300s; medians and inter-quartile ranges over five replicates; logarithmic axis). pyABC falls from  $\approx 8800$  to  $\approx 60$  simulations per second because every generation waits for the slow worker; the asynchronous sampler holds  $\approx 2600$ – $3900$ . The asynchronous run paired with the twin (Table 2) agrees within 15% except at  $1\times$ . (b) Fraction of worker time spent idle under lognormal runtime heterogeneity of spread  $\sigma$  (48 workers, 60s): pyABC idles at its barriers ( $\approx 0.51$  to  $\approx 0.79$ ) while the asynchronous sampler stays near-fully utilized ( $\approx 0.01$  to  $\approx 0.27$ ). (c) Barrier cost predicted from the asynchronous run’s timing (§5.3) against the cost measured with the barrierized twin, for the straggler (16 workers), heterogeneity (48) and Cellular Potts experiments ( $50^3$  at 48–384 workers, utilization ratio, earlier configuration;  $80^3$  at 48, against pyABC). Medians with replicate ranges; the dashed line is equality; open markers are the three configurations outside the model’s domain.

almost nothing ( $1.06\times$ , predicted 1.00): against 1s evaluations the collective’s latency is negligible, and the barrier’s cost is a pure waiting cost. The cost then grows to 2.8, 6.6, 16.6 and  $29.6\times$ , against 2.77, 6.34, 14.0 and  $35.4\times$  predicted from the injected law. The misses, 16% and 19% at the two largest spreads, occur where the twin sees only ten generations per replicate and the 60s budget censors the asynchronous run’s longest draws. pyABC falls 2.1 to  $4.8\times$  behind the asynchronous sampler over the same range, less than the twin, because it closes a generation on the first 100 finishers and discards the latecomers it has already paid for; the comparison against it therefore understates the barrier’s unmitigated cost, and it is the comparison a practitioner faces.

#### *The real simulator.*

On Cellular Potts at  $50^3$  the twin ran at 48, 96, 192 and 384 workers (earlier configuration), and its throughput was  $1.9\times$  below the asynchronous sampler’s at the first three and  $2.8\times$  at the last (Table F5). That factor is not the barrier’s. Throughput

**Table 2** Barrierized twin on the straggler benchmark ( $W = 16$ , five replicates, medians). Throughput in simulations per second; *predicted* as in §5.3, and at  $0\times$  and  $1\times$  the collective’s latency, absent from the model, dominates. Lower block:  $W_1$  of each sampler’s posterior to the analytic posterior at the twin’s simulation count and seeds, at the bandwidth its schedule reached and, last row, re-reported at  $\epsilon_{(100)}$  (§6.4).

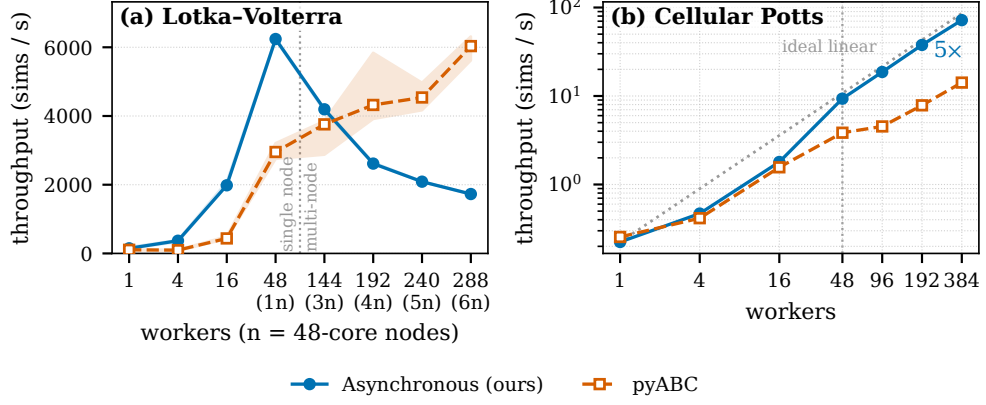
| Straggler slowdown                                                                                    | $0\times$          | $1\times$          | $5\times$   | $10\times$  | $20\times$  |
|-------------------------------------------------------------------------------------------------------|--------------------|--------------------|-------------|-------------|-------------|
| Asynchronous, sims/s                                                                                  | 3888               | 3112               | 3241        | 3218        | 3205        |
| Twin, barrier every $W$                                                                               | 48.7               | 65.3               | 31.6        | 15.9        | 8.0         |
| Twin, barrier every 112                                                                               | 162.8              | 148.4              | 31.8        | 15.9        | 8.0         |
| Twin, predicted                                                                                       | (3870)             | (154)              | 31.8        | 15.9        | 7.98        |
| Ratio, measured (every $W$ )                                                                          | $80\times$         | $48\times$         | $103\times$ | $202\times$ | $402\times$ |
| Ratio, predicted                                                                                      | —                  | —                  | $102\times$ | $202\times$ | $401\times$ |
| <i>Posterior recovery, <math>W_1</math> to the analytic posterior, at the twin’s simulation count</i> |                    |                    |             |             |             |
| Evaluations (= calls per rank $\times 16$ )                                                           | 75,104             | 44,512             | 12,304      | 6,304       | 3,200       |
| Bandwidth reached, asynchronous                                                                       | $5 \times 10^{-4}$ | $7 \times 10^{-4}$ | 0.003       | 0.006       | 0.16        |
| Bandwidth reached, twin every $W$                                                                     | $5 \times 10^{-4}$ | $8 \times 10^{-4}$ | 0.06        | 1.3         | 2.9         |
| $\epsilon_{(100)}$ of the same history                                                                | $3 \times 10^{-4}$ | $5 \times 10^{-4}$ | 0.002       | 0.003       | 0.008       |
| Asynchronous at that count                                                                            | 0.011              | 0.009              | 0.010       | 0.010       | 0.068       |
| Twin, barrier every $W$                                                                               | 0.013              | 0.007              | 0.016       | 0.789       | 1.713       |
| Twin, barrier every 112                                                                               | 0.009              | 0.009              | 0.009       | 0.017       | 1.082       |
| Twin every $W$ , re-reported at $\epsilon_{(100)}$                                                    | 0.015              | 0.010              | 0.010       | 0.010       | 0.010       |

is utilization over mean simulation time, and the two factors separate: the utilization ratio is  $1.2\times$  at every worker count and matches both the straggler factor measured inside the twin’s own generations and the prediction from the asynchronous run’s runtime distribution. The remaining  $1.5\text{--}2.3\times$  is the twin’s simulations taking longer, uncorrelated with the parameters, which we do not attribute and do not count, so the utilization ratio is a lower bound on this workload (Table F5, Appendix F). On this workload the barrier costs  $1.2\times$ , the ceiling its runtime spread sets.

That ceiling rises with the simulator’s size, and the  $80^3$  configuration tests the prediction at a realistic cost and spread. There each evaluation costs 170s, the in-run runtime coefficient of variation is 0.23 on pyABC, and against pyABC the asynchronous sampler’s utilization is 98% against 49% and its throughput is  $2.1\times$  (predicted  $1.90\times$ ). At  $50^3$  the same comparison against pyABC gives  $2.3\times$  on the scaling experiment and  $2.4\times$  on the production run, of which only  $1.2\times$  is the barrier. The rest is pyABC’s fixed per-generation overhead, a large fraction of a 13s evaluation and negligible against a 170s one. We therefore quote  $2.0\times$  for the Cellular Potts systems gain, the number attributable to the mechanism the paper is about. The asynchronous sampler itself scales almost linearly on this workload, at 97% parallel efficiency from one to eight nodes (Fig. 3, right). At the  $80^3$  runtime spread the predicted straggler factor at 384 workers is  $2.1\times$ ; the larger gap against pyABC at that scale at  $50^3$  is, again, overhead.

### ***Against pyABC.***

The twin isolates the barrier; pyABC is what a practitioner would run instead, and the same experiments score it. Under the persistent straggler its throughput falls from



**Fig. 3** Strong scaling at a fixed wall-clock budget against pyABC with population  $\max(100, W)$ . **Left:** Lotka–Volterra (4ms; 180s; medians and inter-quartile ranges over five replicates). **Right:** Cellular Potts  $50^3$ , earlier configuration (5.1s; 1800s; log–log), with linear scaling dotted; the asynchronous sampler reaches 97% parallel efficiency from one node to eight (84% from one worker).

$\approx 8800$  to  $\approx 60$  simulations per second while the asynchronous sampler holds  $\approx 2600$ – $3900$  (Fig. 2a). Under runtime heterogeneity its workers idle 51–79% of their time against 1–27% for ours, and its throughput is 2.1–4.8 $\times$  below ours (Fig. 2b; Table F4, Appendix F). On Cellular Potts it runs at 43–49% utilization against 98–99.8%, and the asynchronous sampler completes 2.1–2.4 $\times$  its simulations in the same budget. It runs with the same kernel and the same bandwidth schedule (§5.2), so none of this is a kernel or schedule difference.

## 6.2 Throughput converts into a tighter tolerance (C2)

Throughput is instrumental; what a practitioner buys is the tolerance the run reaches. Table 3 reads that off every benchmark as  $\epsilon_{(100)}(n)$  (§5.3; the curves are in Appendix F, Fig. F2). Read at the same  $n$  (pyABC’s own count), the asynchronous sampler is more efficient per simulation on three of the four benchmarks (1.06 $\times$  on g-and-k, 1.30 $\times$  on Lotka–Volterra, 1.6 $\times$  on Cellular Potts). On the one-dimensional analytic target it is 0.78 $\times$ , the only benchmark on which it is worse per draw. The ratio rises with the cost of a simulation. The throughput ratio rises with it too, and the two combine into the net effect at equal wall clock, which exceeds one on two benchmarks: 1.3 $\times$  on Lotka–Volterra and 4.1 $\times$  on Cellular Potts. The curves add what a single budget cannot: on the two cheap benchmarks the asynchronous and pyABC curves have the same slope, so the per-simulation ratio is a constant factor. On Cellular Potts, by contrast, both adaptive samplers steepen well beyond the  $n^{-1}$  that rejection ABC follows in this noise-dominated regime, so their advantage over it widens with every doubling of the budget.

**Table 3** Tolerance at a matched budget on every benchmark ( $k = 100$  order statistic of each method’s own discrepancies; five replicates; medians, with the replicate range of each ratio in parentheses; completed simulations only). Per-simulation:  $\epsilon$  of pyABC over the asynchronous sampler at pyABC’s own simulation count; equal wall clock: the same ratio at the end of both runs.

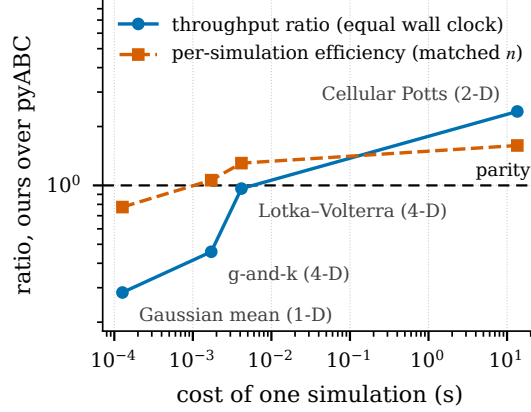
| Benchmark      | cost/sim | sims (async / sync) | throughput | per-simulation    | equal wall clock  |
|----------------|----------|---------------------|------------|-------------------|-------------------|
| Gaussian mean  | 0.1 ms   | 1.31M / 4.61M       | 0.28×      | 0.78× (0.77–1.12) | 0.24× (0.21–0.28) |
| g-and-k        | 2 ms     | 1.75M / 3.81M       | 0.46×      | 1.06× (1.02–1.15) | 0.88× (0.85–0.92) |
| Lotka–Volterra | 4 ms     | 1.51M / 1.56M       | 0.96×      | 1.30× (1.08–1.51) | 1.30× (1.02–1.43) |
| Cellular Potts | 13 s     | 12,900 / 5,400      | 2.39×      | 1.6× (1.14–1.94)  | 4.09× (3.1–6.6)   |

### *Cellular Potts, fixed production run.*

On the real simulator the decomposition is measured on the fixed production run (Table 5). In the same hour the asynchronous sampler completes  $2.4\times$  the simulations of pyABC (99.8% worker utilization against 42.8%; §6.1 attributes  $1.2\times$  of this to the barrier). It reaches a  $1.6\times$  tighter tolerance at pyABC’s own simulation count and ends  $4.1\times$  tighter at equal wall clock ( $7.6 \times 10^{-5}$  against  $3.1 \times 10^{-4}$ ). The per-simulation ratio is read at 5,400 simulations, where pyABC’s curve is the steeper over its last decade ( $n^{-2.2}$  against  $n^{-1.4}$ , Fig. F2d). A fairly resourced rejection sampler (Table 5) reaches  $2.9 \times 10^{-3}$ ,  $39\times$  looser. At  $80^3$  the asynchronous sampler completes  $2.1\times$  the simulations and is  $2.1\times$  more efficient per simulation.

## 6.3 Where the advantage turns on (C3)

The asynchronous sampler pays a roughly fixed cost per arrival: it rebuilds the  $\mathcal{O}(k)$  proposal, weights the arrival against  $S+1$  proposal snapshots, updates the archive, and exchanges the arrival with every other worker. That cost decides where the method pays. Figure 4 plots the throughput and per-simulation ratios of Table 3 against the cost of one simulation. Both are monotone in that cost, but only the throughput ratio has a mechanism: a fixed per-arrival cost against a growing per-simulation one. The per-simulation ratio is a statistical property of four inference problems that differ in dimension, summaries and noise, so the rule rests on the throughput ratio alone. That ratio is  $0.28\times$  at 0.1ms,  $0.46\times$  at 2ms,  $0.96\times$  at 4ms and  $2.4\times$  at 13s, so parity is reached at about 4ms per simulation and the net advantage at equal wall clock turns positive between 2 and 4ms. That boundary belongs to the placement and budget it was measured under. The cheap benchmarks ran 48 ranks spread over four nodes (Appendix D), so every arrival crosses the network; on one packed node the same Lotka–Volterra configuration gives a throughput ratio of 2.1 instead of 0.96 (Table F6). The crossover moves down with tighter placement. Below it lie the two cheap benchmarks, measured on the wrong side of the line. On the Gaussian mean the asynchronous sampler finishes with a tolerance  $4\times$  looser than pyABC’s at equal wall clock. Per simulation it is  $0.78\times$  as efficient; most of the gap comes from pyABC completing  $3.5\times$  more simulations of a 0.1 ms simulator. Where the simulations are that cheap a barrier costs nothing worth removing.



**Fig. 4** Throughput ratio and per-simulation efficiency of the asynchronous sampler over pyABC, against the cost of one simulation, one point per benchmark (Table 3). Both rise with the cost; throughput parity is reached at about 4 ms per simulation at these budgets and this placement (§6.3).

The same cost sets the scaling boundary on a cheap simulator. On Lotka–Volterra the exchange of every arrival with every worker grows with the worker count: a worker is inside the simulator 25% of its time at 48 workers and 2% at six nodes. Throughput peaks on one node, and pyABC with a population equal to the worker count overtakes beyond three nodes (Fig. 3 left; Appendix F). At 13 s per evaluation the same coordination is negligible, and the method tracks linear scaling to eight nodes (Fig. 3, right).

## 6.4 The posterior: calibration, recovery, and what limits it (C4)

### Calibration.

Table 4 gives simulation-based calibration of the full-history estimator (1000 trials, matched simulation budget). On the one-dimensional Gaussian mean it tracks the nominal coverage at every level, while pyABC under-covers at every level (its nominal 0.90 interval covers 0.75); the Monte Carlo error of  $\pm 0.01$  resolves this ordering. On a bimodal target ( $y_i \sim \mathcal{N}(\theta^2, \sigma^2)$ , the case a top- $k$  archive is most at risk of collapsing onto one mode) the archive retains both modes in 86% of trials and coverage is nominal. On the four-dimensional g-and-k model the estimator is *conservative* on  $A$  and  $g$  and near nominal on  $B$  and  $k$ , with  $B$ ’s rank histogram the only uniform one at the 1% level. pyABC is near nominal on  $A$  and  $g$  and under-covers  $B$  and  $k$ , with no uniform rank histogram. The evidence points to a reporting-support effect rather than a weighting failure: the top- $k$  archive alone would under-cover (0.85/0.89 at the 0.90/0.95 levels), the full history over-covers, and coverage rises monotonically through nominal as the support grows from one to the other. Doubling the simulation budget leaves the coverage unchanged. A sweep over the archive size and snapshot buffer on a  $d$ -dimensional Gaussian target (Appendix F) finds a miscalibrated region at  $d = 8$ ,  $k = 50$ ; of the archive sizes swept,  $k = 100$  is the best calibrated.

**Table 4** Simulation-based calibration of the posterior estimate (1000 trials; empirical coverage of central intervals at the nominal levels; Monte Carlo standard error  $\pm 0.007$ – $0.016$ ). Nominal is best.

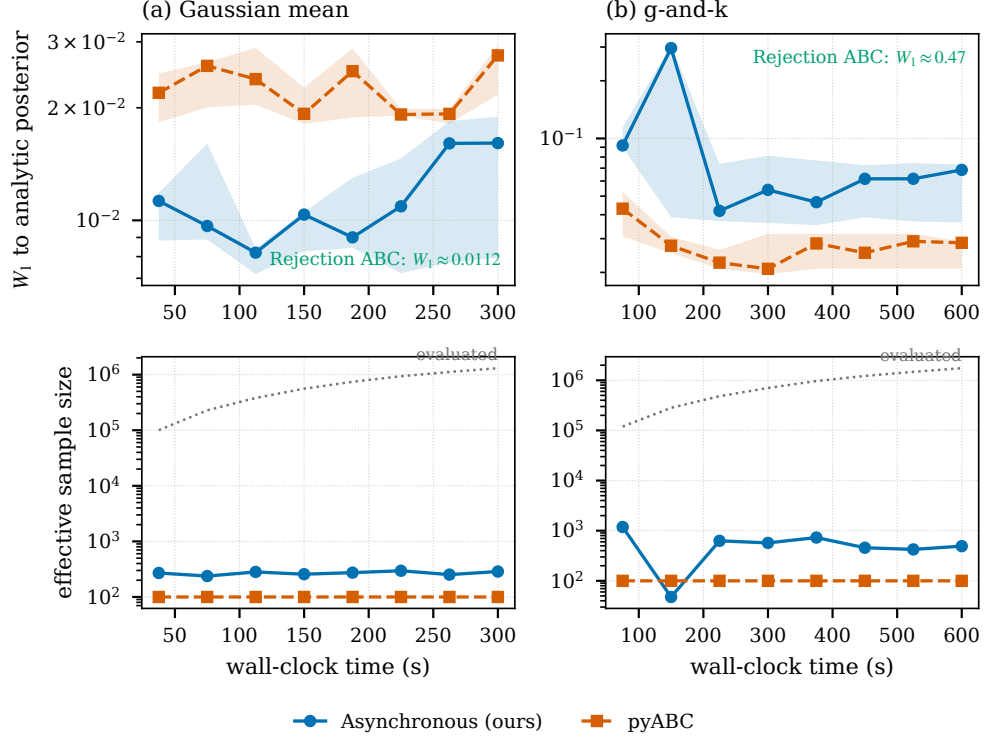
| Target                                          | Method                         | 0.50 | 0.80 | 0.90 | 0.95 |
|-------------------------------------------------|--------------------------------|------|------|------|------|
| Gaussian mean (1-D)                             | asynchronous                   | 0.51 | 0.80 | 0.90 | 0.94 |
|                                                 | pyABC                          | 0.35 | 0.63 | 0.75 | 0.83 |
| Bimodal $\mathcal{N}(\theta^2, \sigma^2)$ (1-D) | asynchronous                   | 0.50 | 0.82 | 0.91 | 0.96 |
|                                                 | pyABC                          | 0.45 | 0.82 | 0.90 | 0.95 |
| g-and-k (4-D), $A$                              | asynchronous, full history     | 0.62 | 0.91 | 0.96 | 0.99 |
|                                                 | pyABC                          | 0.54 | 0.82 | 0.91 | 0.95 |
| g-and-k (4-D), $B$                              | asynchronous, full history     | 0.54 | 0.83 | 0.91 | 0.96 |
|                                                 | pyABC                          | 0.45 | 0.74 | 0.85 | 0.90 |
| g-and-k (4-D), $g$                              | asynchronous, full history     | 0.58 | 0.88 | 0.95 | 0.98 |
|                                                 | pyABC                          | 0.51 | 0.82 | 0.91 | 0.95 |
| g-and-k (4-D), $k$                              | asynchronous, full history     | 0.56 | 0.83 | 0.92 | 0.96 |
|                                                 | pyABC                          | 0.47 | 0.74 | 0.86 | 0.92 |
| g-and-k (4-D), $A$                              | asynchronous, top- $k$ archive | —    | —    | 0.85 | 0.89 |

### *Recovery against a reference posterior.*

Where a reference exists, Fig. 5 scores each method’s posterior against it, replayed over wall-clock prefixes. On the Gaussian mean the asynchronous estimator sits at  $W_1 = 0.008$ – $0.016$  from the analytic posterior, closer than pyABC’s final population ( $0.019$ – $0.028$ ) at every checkpoint; a fairly budgeted rejection sampler’s best 100 of 20,000 draws sits at 0.011. In one dimension with that many draws, rejection is as good. On the four-dimensional g-and-k the order reverses: pyABC ends at 0.029, the asynchronous estimator at 0.069, rejection at 0.47. The lower panels say why. The posterior’s effective sample size is 238–296 at every checkpoint of the Gaussian run while the history grows from  $10^5$  to  $1.3 \times 10^6$  particles. A controlled sweep puts ESS/ $k$  between 1.9 and 3.6 over  $k \in \{50, 100, 200\}$  and  $n$  up to  $8 \times 10^4$ . The estimator reports at the tightest bandwidth reached, and that bandwidth is set by ESS-retention bisection against the archive, so the effective support stays at a few multiples of  $k$  however many particles are evaluated. In one dimension that support is small and stable. In four it is small and erratic (7 to 2987 across checkpoints and replicates), and accuracy follows it. The four-dimensional deficit is consistent with intermittent weight degeneracy rather than bias, and the ceiling binds when the target is identified well enough for the bandwidth to tighten.

### *A known truth on the real simulator.*

On the two-parameter Cellular Potts setup (one synthetic dataset with a known truth), all three methods recover the truth (Fig. 6, Table 5). The division rate is well identified by every method (91–92% contraction against the prior). The cell volume separates them:  $81\% \pm 1\%$  for the asynchronous estimator,  $72\% \pm 4\%$  for pyABC’s last generation with its population weights, and 67% for the best 100 rejection draws. The truth lies inside the central 90% interval of every run (five per adaptive method, one rejection run); on a single dataset that is containment, not coverage. The asynchronous

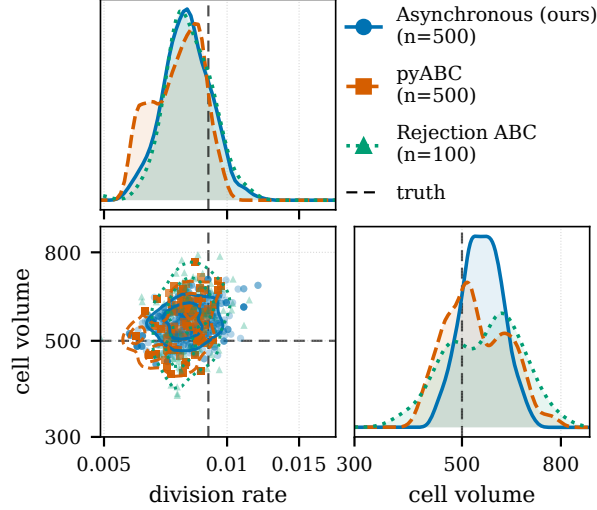


**Fig. 5** What the estimator recovers, and with how many effective particles. Each method’s own posterior (§5.3) is scored against the analytic posterior (a) and the g-and-k reference posterior (b). Top:  $W_1$ , median and inter-quartile range over five replicates. Bottom: the effective sample size of the posterior estimate, with the evaluated-particle count dotted; the history grows by more than an order of magnitude and the effective sample does not follow it.

estimator is the tightest and carries the largest effective sample (303 against pyABC’s 66 of 100, medians).

### *The starting bandwidth.*

The first row of Table 5 is the same sampler with the starting bandwidth the benchmark shipped with,  $\epsilon_0 = 10$ . It reports  $13\% \pm 14\%$  contraction on the cell volume, which is erratic as well as poor. The sampler’s draws are comparably concentrated in both rows (the end tolerance is  $6.3 \times 10^{-5}$  against  $7.6 \times 10^{-5}$ ); what differs is the bandwidth the estimator reports at. The schedule tightens at the bounded rate of §3.2, so on an expensive simulator that transient is the whole run. The run’s 3000 evaluations on 48 ranks are 62 calls per rank; at  $\epsilon_0 = 10$  the bandwidth had not moved after them and ended  $321\times$  above the tolerance the run’s own draws supported, while at 0.1 it ended  $2\times$  above it. On a simulator that affords millions of evaluations the transient is negligible, which is why the cheap production runs never showed it. It binds in the regime the method is designed for. The prior-predictive median discrepancy can be



**Fig. 6** Cellular Potts posterior (division rate, cell volume) on the fixed production run, replicate 0: the asynchronous AMIS estimator (500 resampled draws), pyABC’s last completed generation with its population weights (500 resampled draws), and the 100 best rejection draws (Table 5). Axes are the log-uniform prior coordinates ticked in physical units; dashed lines mark the truth. Every method recovers the division rate; the asynchronous estimator is the tightest on the cell volume.

**Table 5** Cellular Potts  $50^3$ , five replicates per adaptive method and one rejection run, 3600s on 48 workers (means  $\pm$  s.d. for contraction, medians otherwise). The first row is the asynchronous sampler with the shipped starting bandwidth (10, twenty times the prior-predictive median discrepancy), the rest the production configuration (0.1). Rejection ABC is one run of 13,000 prior draws on all 48 ranks, keeping the best 100. Contraction is  $1 - \text{sd}/\text{sd}_{\text{prior}}$ ; the truth lies inside the central 90% interval of every run.

|                                   | sims   | util. | $\epsilon_{(100)}, \text{end}$ | contraction   |               | ESS |
|-----------------------------------|--------|-------|--------------------------------|---------------|---------------|-----|
|                                   |        |       |                                | division rate | cell volume   |     |
| Asynchronous, $\epsilon_0 = 10$   | 13,300 | 99.8% | $6.3 \times 10^{-5}$           | $84 \pm 6\%$  | $13 \pm 14\%$ | 645 |
| Asynchronous, $\epsilon_0 = 0.1$  | 12,900 | 99.8% | $7.6 \times 10^{-5}$           | $92 \pm 1\%$  | $81 \pm 1\%$  | 303 |
| pyABC, $\epsilon_0 = 0.1$         | 5,400  | 42.8% | $3.1 \times 10^{-4}$           | $91 \pm 1\%$  | $72 \pm 4\%$  | 66  |
| Rejection ABC, best 100 of 13,000 | 13,000 | —     | $2.9 \times 10^{-3}$           | 91%           | 67%           | 100 |

measured before the run and without the truth (0.46 here, from 2000 prior draws), and setting  $\epsilon_0$  to about a fifth of it recovers the posterior. Re-reporting a stored history at the  $k$ -th order statistic of its own losses, at no simulation cost, reaches a comparable posterior (93%/85% on replicate 0) and is the check we apply.

### *The schedule must move.*

The barrierized twin shows the same transient on a cheap simulator. At the twin’s own simulation count and seeds the asynchronous sampler reports  $W_1 = 0.010$  and 0.068 to the analytic posterior at  $10\times$  and  $20\times$  where the twin reports 0.79 and 1.71

(Table 2, lower block). The propagator is the same, and so are the 200 calls per rank at  $20\times$ . But the twin’s schedule reached a bandwidth of 2.9, against 0.16, on histories whose 100th-smallest discrepancy is 0.008, because under lockstep it takes fewer search steps (Appendix C). Re-reporting the twin’s histories at  $\epsilon_{(100)}$  brings them to 0.010–0.015. The estimate is only as good as the bandwidth the schedule reached, and the order-statistic re-report is the check.

### *The archive size.*

The effective sample size is a few multiples of  $k$ , but raising  $k$  is not free: re-reporting one history at the order statistic for  $k = 10$  to 1000 loosens the final bandwidth and moves the cell-volume contraction from 85% to 78%; running with  $k = 300$  leaves the schedule at  $\epsilon_0$  for the whole hour (7% contraction); and on the cheap benchmarks  $k = 100$  is Pareto-optimal for throughput and calibration (Appendix F). The archive size is therefore a benchmark-dependent choice, made by checking calibration and the bandwidth reached.

## 7 Discussion

Against pyABC with the same kernel and schedule, the method is ahead on the tissue simulator in simulations per hour ( $2.4\times$ ), tolerance at equal wall clock ( $4.1\times$ ), contraction on the weakly identified parameter (81% against 72%) and effective sample size (303 against 66). On the synthetic targets it is calibrated wherever pyABC under-covers, and it loses only where a simulation costs less than a few milliseconds. The results give a practitioner four rules, each backed by a measurement.

*Use it above a few milliseconds per simulation.* Below that the per-arrival coordination of a single shared archive costs more than a generation barrier does, and a generation-staged sampler with a population sized to the machine is the better tool (§6.3). Above it the barrier is the larger cost, and removing it recovers that cost, which grows with the worker count and with the spread of simulation times. A simulator’s spread can grow with its size, as the Cellular Potts model’s did.

*Predict the gain before paying for it.* A production project with an unknown posterior cannot afford a synchronous reference run to find out what asynchrony would have bought. It does not need one: the barrier’s cost is  $W/\mathbb{E}[\max_{i \leq W} D_i]$  against the asynchronous throughput, computable from the run’s own per-evaluation timing (§6.1). Two cautions come with it. The timing sample must be longer than the tail of the runtime distribution, or the prediction undershoots; and a persistently slow worker contributes its own evaluation time, not its wall-clock span.

*Set the starting bandwidth from the prior-predictive discrepancy scale.* On an expensive simulator the whole run can be spent inside the bandwidth transient, and a starting value twenty times the prior-predictive median costs the posterior most of its contraction on the weakly identified parameter while the draws stay comparably concentrated (§6.4). About a fifth of the median discrepancy of a few thousand prior draws is the rule that transferred. The check is to re-report a stored history at the order statistic of its own losses and to confirm that the schedule moved at all.

*Keep the archive small and check calibration.* The effective sample size is a few multiples of the archive size, and enlarging the archive loosens the final bandwidth and lengthens the transient. An archive of  $k = 100$  was never far from the best choice on any benchmark, and the calibration checks of §6.4 are cheap to repeat.

The effective-sample-size ceiling is where the method has headroom. Barrier removal delivers simulations reliably and predictably. Converting all of them into posterior accuracy is limited by the reporting rule, which we adopted for its simplicity: report at the tightest bandwidth reached, chosen by ESS retention against the archive. Further work on this method belongs in that rule rather than in the scheduler.

## 8 Limitations

Consistency holds up to the fidelity ratio  $r$ , of which two factors are measured and two bounded, with the adapted filtration under asynchronous execution assumed (§4); the central limit theorem is proved for damped-adaptation variants, and the production runs sit at the threshold of its rate condition rather than clearly inside it (Appendix A). Keeping every evaluated particle over-represents parameter regions that simulate quickly, the length bias of deadline-limited sampling [30]; a runtime-coupled study found no measurable shift of the posterior mean at the couplings tested, and we do not correct for it (Appendix F).

## 9 Conclusion

A generation barrier costs the ratio of the barrier-free throughput to what a generation allows. That cost is predictable from the asynchronous run’s timing alone: the prediction matched a barrierized twin of our own sampler on synthetic runtime laws and on a real tissue simulator. The generation-free sampler we propose removes the barrier and, on our benchmarks, converts the simulations it gains into a tighter tolerance wherever a simulation costs more than a few milliseconds: on the tissue simulator,  $2.4\times$  the simulations of a kernel-matched pyABC in the same hour, a  $4.1\times$  tighter tolerance, and a tighter posterior on a larger effective sample. Its estimator is consistent up to a partly measured fidelity ratio, calibrated in one dimension and conservative in four, and replayable from the log. What bounds it is the bandwidth it reaches: the starting bandwidth must be set from the prior-predictive scale, the schedule must be allowed to move, and the effective sample size is a few multiples of the archive size. For simulator-based inference on machines where simulations are expensive and unequal, our results support generation-free execution. A practitioner adopting it has what the choice needs: a prediction of the gain from their own timing, and the two settings on which the posterior depends.

**Acknowledgments.** *To be added in the final version.*

## Declarations

### Funding

*To be added in the final version.*

**Conflict of interest/Competing interests**

The authors declare no competing interests.

**Ethics approval and consent to participate**

Not applicable.

**Consent for publication**

Not applicable.

**Data availability**

The repository accompanying this paper contains the summary data behind every figure and table, the scripts that regenerate each from the experiment output, and the committed output of every diagnostic quoted in the text, so every figure, table and quoted number can be regenerated from the repository alone. The raw per-particle records ( $\approx 30$  GB for the Gaussian-mean experiment alone), the per-rank timing logs and the resolved run configurations will be deposited in a public archive upon publication.

**Materials availability**

Not applicable.

**Code availability**

The full implementation (the Propulate propagator, the matched-kernel pyABC baseline, the analysis and plotting scripts, and the test suite) will be released in a public repository upon publication.

**Author contribution**

*To be added in the final version.*

## Appendix A The central limit theorem and the fidelity ratio

This appendix carries the central limit theorem and its per-run confidence intervals, a bound on how far the tilt by  $r$  can move the target, and the decomposition and partial measurement of  $r$  on stored runs.

***Relation to existing AMIS theory.***

For a fixed proposal, the  $r$ -tilted limit of Theorem 1 is the elementary observation that self-normalized importance sampling against a mismatched denominator converges to a correspondingly tilted target. Three things are not routine. The scheme is adaptive at every draw, so the argument runs through a martingale array rather than an i.i.d. one, and the available AMIS results do not cover it: [6]’s convergence argument is heuristic, and [17] restricts the adaptation so that the weighting stage decouples, which a single-arrival scheme does not do. The prior floor in the *denominator* bounds every weight by  $1/\delta$  with no moment or density-ratio condition. This makes the conditional Lindeberg step trivial and lets the method use a local archive proposal at all, for which the usual two-sided ratio bound fails. And the departure from the ideal is carried as  $r$  inside the statements, then decomposed and partly measured.

**Assumption 5(ii), in full.**

In addition to clause (i),  $\epsilon_\infty$  and  $r$  are deterministic, a (possibly random) limit  $\tilde{q}_\infty$  of  $\tilde{q}_i$  and  $\bar{q}_\infty$  of  $\bar{q}_n$  exist with  $r = \tilde{q}_\infty/\bar{q}_\infty$ , and, almost surely,

$$\begin{aligned} \sqrt{n} (|\epsilon_n - \epsilon_\infty| + \|\bar{q}_n - \bar{q}_\infty\|_\infty) &\rightarrow 0, \\ \frac{1}{\sqrt{n}} \sum_{i \leq n} (|\epsilon_{i-1} - \epsilon_\infty| + \|\tilde{q}_i - \tilde{q}_\infty\|_\infty + \|\bar{q}_{i-1} - \bar{q}_\infty\|_\infty) &\rightarrow 0, \end{aligned} \quad (\text{A1})$$

where  $\epsilon_{i-1}$  and  $\bar{q}_{i-1}$  are the bandwidth and the denominator (6) formed from the first  $i-1$  draws (both known before draw  $i$ ; convention  $\bar{q}_0 := \pi$ ). The two clauses differ in strength: consistency permits  $r$  and  $\epsilon_\infty$  to be random, while the central limit theorem needs them deterministic, because the centered integrand must be a fixed function for the martingale arrays to be adapted. That restriction costs nothing in the case of most interest,  $r \equiv 1$ . The Lipschitz constant of Assumption 1 is stated per compact sub-interval because it degrades as  $\epsilon \downarrow 0$ ; no deterministic bandwidth floor is needed, since the proofs localize over  $[1/j, \epsilon_0]$  and intersect countably many almost-sure events (Appendix B).

**Theorem 3** (central limit theorem). *Let Assumptions 1–4 and 5(i)–(ii) hold. Then for every bounded measurable  $f$ ,*

$$\sqrt{n}(\hat{\pi}_n(f) - \pi_{\epsilon_\infty}^r(f)) \longrightarrow \sigma_f \mathcal{Z} \quad \text{stably}, \quad \mathcal{Z} \sim \mathcal{N}(0, 1) \text{ independent of } \mathcal{F}_\infty, \quad (\text{A2})$$

where the  $\mathcal{F}_\infty$ -measurable  $\sigma_f^2$  is

$$\sigma_f^2 = \frac{1}{Z_r^2} \int_{\Theta} \pi(\theta)^2 (f(\theta) - \pi_{\epsilon_\infty}^r(f))^2 L_{\epsilon_\infty}^{(2)}(\theta) \frac{\tilde{q}_\infty(\theta)}{\bar{q}_\infty(\theta)^2} d\theta, \quad Z_r = \int_{\Theta} \pi L_{\epsilon_\infty} r. \quad (\text{A3})$$

Theorem 3 covers variants whose adaptation is frozen after a finite burn-in or damped fast enough for the rate condition (A1); whether the data-driven schedule our experiments run meets that condition is measured in §A.1, where the production runs sit at its threshold. At  $r \equiv 1$  the variance collapses to  $Z_{\epsilon_\infty}^{-2} \int \pi^2 (f - \pi_{\epsilon_\infty} f)^2 L_{\epsilon_\infty}^{(2)} / \bar{q}_\infty$ , the familiar self-normalized form. Because  $L_\epsilon^{(2)} \geq L_\epsilon^2$  by Jensen, (A3) is never smaller than the variance of the corresponding *noise-free* importance sampler that could evaluate  $L_{\epsilon_\infty}$  exactly; the gap  $L_\epsilon^{(2)} - L_\epsilon^2 = \text{Var}(K_\epsilon(\rho) \mid \theta)$  is the price of running one simulation per proposed parameter. For the Gaussian kernel  $K_\epsilon(\rho)^2 = K_{\epsilon/\sqrt{2}}(\rho)$ , so  $L_\epsilon^{(2)} = L_{\epsilon/\sqrt{2}}$  and that price is computable from the same likelihood at a tightened bandwidth. Convergence is stable rather than merely distributional because  $\tilde{q}_\infty$ , and hence  $\sigma_f^2$ , may be random; it reduces to an ordinary  $\mathcal{N}(0, \sigma_f^2)$  limit when the limiting proposal is deterministic. Stability is also what makes the theorem usable, because it survives division by a consistent variance estimate:

**Corollary 4** (per-run confidence intervals). *Under the assumptions of Theorem 3, and provided  $\sigma_f^2 > 0$ , with  $\hat{\sigma}_n^2 := n \sum_i W_{i,n}^2 (f(\theta_i) - \hat{\pi}_n(f))^2 / (\sum_i W_{i,n})^2$  we have  $\hat{\sigma}_n^2 \rightarrow \sigma_f^2$  a.s. and  $\sqrt{n}(\hat{\pi}_n(f) - \pi_{\epsilon_\infty}^r(f)) / \hat{\sigma}_n \Rightarrow \mathcal{N}(0, 1)$ . The interval  $\hat{\pi}_n(f) \pm z_{1-\alpha/2} \hat{\sigma}_n / \sqrt{n}$*

therefore has asymptotic level  $1 - \alpha$  and is computable from the stored weights alone, at no extra simulation cost.

**Proposition 5** (how far the tilt can move the target). *Let  $A \subseteq \Theta$  and suppose  $|r - 1| \leq \eta_A$  on  $A$  and  $|r - 1| \leq 1$  off it. Then with  $\zeta := \eta_A + \pi_{\epsilon_\infty}(A^c)$ ,*

$$\zeta < 1 \implies \|\pi_{\epsilon_\infty}^r - \pi_{\epsilon_\infty}\|_{\text{TV}} \leq \frac{\zeta}{1 - \zeta}. \quad (\text{A4})$$

More sharply,  $\zeta$  may be replaced by  $\zeta_0 = \pi_{\epsilon_\infty}(|r - 1|)$ . Estimating  $\zeta_0$  from a stored run requires untilting: the normalized weights  $\hat{w}_i$  target  $\pi^r$ , not  $\pi_{\epsilon_\infty}$ , so  $\sum_i \hat{w}_i |r(\theta_i) - 1|$  estimates  $\pi^r(|r - 1|)$ ; weights proportional to  $W_{i,n}/r(\theta_i)$  target  $\pi_{\epsilon_\infty}$  and give  $\hat{\zeta}_0$ .

## A.1 What $r$ is for this implementation

Everything specific to the code enters through  $r$ . To attribute it, walk from the density actually sampled from down to the function actually divided by, changing one thing at a time. Let  $\bar{q}^{(1)} = \frac{1}{n} \sum_{i \leq n} q_i$  use the *nominal* archive proposals (the truncated mixtures the propagator intends, with exact normalizing constants and without the rejection cap or the underflow redraws); let  $\bar{q}^{(2)}$  replace those constants by the implementation's product-of-marginals values; and let  $\bar{q}^{(3)}$  be the  $m$ -point draw-proportional version of  $\bar{q}^{(2)}$  carrying the observed bootstrap share  $\nu_n$ . Assume each of the four displayed ratios has a uniform limit, finite and bounded away from zero. Fidelity alone does not give this, since (8) constrains only their product. Writing  $\bar{q}$  for the  $m$ -snapshot denominator (6),

$$r = \underbrace{\lim \frac{\bar{q}^*}{\bar{q}^{(1)}}}_{\text{(d) cap, redraws, asynchrony}} \cdot \underbrace{\lim \frac{\bar{q}^{(1)}}{\bar{q}^{(2)}}}_{\text{(c) normalizer}} \cdot \underbrace{\lim \frac{\bar{q}^{(2)}}{\bar{q}^{(3)}}}_{\text{(b) quadrature}} \cdot \underbrace{\lim \frac{\bar{q}^{(3)}}{\bar{q}}}_{\text{(a) floor}}.$$

The product telescopes for any positive intermediates, so the content is in the choice of intermediates: a different insertion order attributes the total differently, and the factors are a decomposition of one quantity rather than four independent errors. Under asynchronous execution the first factor also carries the gap between the density the propagator emitted and the conditional law of Assumption 4, so it is partly a property of the schedule.

(a) *The prior floor.* When genuine bootstrap draws carry less than a  $\delta$  share of the history,  $w_n = \delta > \nu_n$  and the denominator holds more prior mass than the sample does. With  $u = q_\infty/\pi$  the limiting proposal-to-prior ratio, this contributes  $r_{\text{floor}} = (\nu_\infty + (1 - \nu_\infty)u)/(\delta + (1 - \delta)u)$ , so  $r_{\text{floor}} - 1 = (\delta - \nu_\infty)(u - 1)/(\delta + (1 - \delta)u)$ : it is exactly 1 when  $\nu_\infty \geq \delta$ , below 1 wherever the proposal is thinner than the prior, and  $r_{\text{floor}} \rightarrow 1$  as  $\delta \rightarrow 0$ , i.e. as  $m$  grows. This is the one contribution available in closed form.

(b) *The snapshot quadrature.* The  $m$ -point mixture stands in for the  $n$ -point one, with error at most  $V_n/m$  where  $V_n = \sum_{i < n} \|q_{i+1} - q_i\|_\infty$  is the total variation of the

*reconstructed* proposal path; this controls the third factor above, not the fidelity of the whole. Fixing  $m$  therefore trades cost against fidelity; see the remark below.

(c) *The truncation constant.* Each mixture component is normalized by the product of its per-axis in-box masses, which is exact for a diagonal perturbation covariance and increasingly accurate as a component’s mass sits away from the faces of the box relative to its own scale, but an approximation otherwise.

(d) *Capped rejection, underflow redraws, and (asynchronously) the filtration gap.* A parent’s perturbation is rejection-sampled into the box for at most 992 attempts, so component  $j$  is realized with probability  $W_j[1 - (1 - Z_j)^{992}]$  and the residual is emitted as a prior draw; and a candidate whose proposal-time denominator falls below  $10^{-12}$  is redrawn up to five times, the last one being emitted regardless. The cap changes the law relative to unlimited rejection whenever  $Z_j < 1$ , through the residual probability  $(1 - Z_j)^{992}$ ;  $Z_j \approx 10^{-3}$  is the scale below which that residual becomes appreciable, not a threshold above which it vanishes. Under asynchronous execution this factor additionally carries the gap between the emitted proposal density and the conditional law of Assumption 4, which is a property of the schedule.

### *Measured, in part.*

Two of the four factors, the prior floor and the snapshot quadrature, can be estimated rather than bounded, because  $\hat{\pi}_n$  is a post-hoc pass: recomputing (6) at a much larger  $m$  and at the observed prior share gives a reference against which those factors are evaluated at every particle. The truncation constant and factor (d) are bounded rather than measured. On the five Cellular Potts production replicates (about 12,900 asynchronous evaluations each across 48 workers, two correlated dimensions, so that (c) is live; Appendix E) both factors are active: the bootstrap draws make up 1.1% of each history against a floor of  $\delta = 2.4\%$ , so (a) contributes  $\hat{\zeta} = 0.011$  and (b) 0.024–0.028, together 0.035–0.039 with  $r \in [0.24, 1.41]$ . The posterior estimate sits 0.2–0.5% in total variation from the reference at  $m = 400$  and the observed prior share, with posterior means within 0.012 standard deviations, marginal widths within 1% (narrower in nine of ten, the tail-suppressing direction that (a) predicts), and the effective sample size unchanged. On a one-dimensional Gaussian-mean history where the exact  $n$ -point mixture is computable, (a) and (b) together give  $\hat{\zeta} = 0.042$  and a total-variation distance of 0.26% from the reference that removes (a) and (b); there too the estimate is 0.5% narrower. Of contribution (d), the two events a stored history records, a rejection fallback and an exhausted redraw, occurred neither in the 12,000 draws of that history nor in the 63,699 archive-phase draws of the five Cellular Potts replicates; a redraw that succeeded on a later attempt leaves no trace, so this is a necessary condition for the event, not a bound on its probability. The bound (A4) is loose against these direct measurements by an order of magnitude on both (3.6–4.0% against 0.2–0.5% on Cellular Potts, 4.4% against 0.26% on the one-dimensional history), as sup-norm-free bounds valid for every  $f$  generally are.

### *Do the reported runs meet the stabilization conditions?*

Assumption 5(ii) is the one place where we can do better than assume, because the proposal sequence is reconstructible from a stored history. Rebuilding  $q_\tau$  along the

five Cellular Potts production replicates and comparing against each run’s terminal proposal separates two rates that behave differently. The *per-step drift*  $\|q_{\tau+1} - q_\tau\|_\infty$  decays as  $\tau^{-0.7}$  to  $\tau^{-0.95}$ , and the archive’s membership changes only 825–874 times in about 12,900 draws, tracking  $k \log n$  at a ratio between 0.54 and 0.92 across the whole run. This is the rate a record-statistics heuristic predicts, since the  $k$ -th best of  $n$  draws improves with probability  $\approx k/n$ . The proposal path’s total variation is therefore  $O(\log n)$ , and the snapshot clause is comfortably met. The *distance to the limit* is the binding one. Fitting  $\|q_\tau - q_\infty\|_\infty \sim \tau^{-b}$  gives  $b = 0.50$ – $0.71$  per replicate over the full range (0.59 pooled) and  $b = 0.25$ – $0.72$  (0.48 pooled) when the fit is restricted to the first half, away from the end where using the terminal proposal as a stand-in for  $q_\infty$  biases  $b$  upward. Assumption 5(ii) needs  $b > \frac{1}{2}$ , at which the aggregate  $n^{-1/2} \sum_{i \leq n} \|q_i - q_\infty\|_\infty$  vanishes like  $n^{1/2-b}$ : the production runs sit at that threshold.

The measurement spans one benchmark over 1.3 decades of  $n$  per replicate, against a stand-in limit, so it is a diagnostic rather than an asymptotic statement. Theorem 3 is accordingly stated for the damped and frozen variants; Theorem 1, needing no rate, is unaffected. What the exponent turns on is the inference problem, not the schedule. A tightening bandwidth does not move it: running a two-dimensional Gaussian-mean benchmark serially under the same propagator, once with the data-driven schedule and once with the bandwidth pinned at the tolerance the first run reached a third of the way in, moves  $b$  by less than 0.07, and both runs satisfy  $b > \frac{1}{2}$  at  $b = 0.61$ – $0.76$ .

Identifiability does. Re-running the serial benchmark with its two parameters confounded (observing only  $\theta_1 + \theta_2$ , so the target is a ridge rather than a point) collapses the exponent to  $b = 0.00$ – $0.29$ : on a ridge the top- $k$  archive slides along the flat direction indefinitely, so the proposal never settles. Assumption 5(ii) is thus a condition on the inference problem as much as on the sampler. It holds with room to spare for a point-identified target under the same top- $k$  rule and bandwidth schedule and fails on a ridge; the Cellular Potts target, which resolves two directions but the second less sharply than the first (Appendix E), lands between the two.

#### ***Fixed versus growing $m$ .***

Contribution (b) is where the estimator’s cost meets its fidelity. With  $m$  fixed, the  $m$ -point mixture tracks the  $n$ -point one only if the proposal path settles, the error being  $V_n/m$ . Letting  $m = m_n \rightarrow \infty$  with  $V_n/m_n \rightarrow 0$  removes *this* contribution to  $r$  (which does not by itself give  $r \equiv 1$ , since the other three factors remain), and the proofs go through for any  $m_n = o(n/\log n)$  *provided the floor  $\delta$  is held fixed rather than tied to  $m$* . The implementation ties them,  $\delta = 0.5/(m+1)$ , so growing  $m$  there would send  $\delta \rightarrow 0$ , and with it the uniform weight bound  $1/\delta$  of Lemma 6 that makes the Lindeberg step trivial and fixes the constants in the law of large numbers; the rate conditions would have to be redone. Decoupling the two is a one-line change to the post-hoc pass (Appendix B, Remark 2); the retroactive pass then costs  $\mathcal{O}(n m_n k)$  instead of  $\mathcal{O}(nk)$ , and the literal cumulative mixture is the endpoint  $m_n = n$  at  $\mathcal{O}(n^2 k)$ . The measurements above locate the practical knee: on the Gaussian-mean history the posterior estimate is within  $3 \times 10^{-4}$  in total variation of the exact  $n$ -point mixture by  $m = 50$ , so a  $2.5\times$  more expensive post-hoc pass removes factor (b) almost entirely. What the paper reports is the  $m = 21$  end, and §3.4 gives its cost.

## Appendix B Proofs

### B.1 Setting

We work with a filtration  $(\mathcal{F}_i)$  for which, writing  $\mathbb{E}_{i-1}[\cdot] = \mathbb{E}[\cdot \mid \mathcal{F}_{i-1}]$ , the pair  $\xi_i = (\theta_i, \rho_i)$  is  $\mathcal{F}_i$ -measurable and the two conditional laws below hold. Particles are indexed by *proposal* time, not arrival time. This is what makes the martingale structure survive asynchronous execution: a worker forms its proposal from the history it has received at that moment, which is part of  $\mathcal{F}_{i-1}$ , so the conditional density  $\tilde{q}_i$  of the emitted candidate is  $\mathcal{F}_{i-1}$ -measurable and

$$\theta_i \mid \mathcal{F}_{i-1} \sim \tilde{q}_i, \quad \rho_i \mid \mathcal{F}_{i-1} \vee \sigma(\theta_i) \sim p(\cdot \mid \theta_i). \quad (\text{B5})$$

The proofs index by *proposal* time, whereas the implementation's post-hoc estimator consumes a rank's *arrival*-ordered log and reconstructs the  $q_{\tau_s}$  from prefixes of it; the two agree for a sequential run and need not otherwise, and the discrepancy is one more contributor to  $r$ . Under sequential execution  $\mathcal{F}_i = \sigma(\xi_1, \dots, \xi_i)$  serves and (B5) holds by construction. Under asynchronous execution a discrepancy  $\rho_i$  may still be in flight when candidate  $i + 1$  is proposed, so the  $\sigma$ -field of everything realized by that moment does not contain  $\xi_i$ . One must take a filtration large enough to contain the in-flight outcomes, and Assumption 4 asserts that such a filtration exists *with  $\tilde{q}_i$  still the conditional law of the candidate under it*. That is the substantive clause. Conditioning on the larger field need not leave the candidate's law unchanged when completion times steer which rank proposes next. In that case  $\tilde{q}_i$  is the enlarged-field conditional law rather than the propagator's emitted density, and (8) is a statement about that object: the completion-time selection of §8, seen from the theory side. Concurrency also affects the reconstruction of the  $q_{\tau_s}$  from a single rank's arrival-ordered log, which enters only through  $\|\bar{q}_n - \bar{q}_\infty\|_\infty$  in Assumption 5.

No structural form is imposed on  $\tilde{q}_i$ : it is whatever the archive rule, the capped rejection, the underflow redraws and the bootstrap phase jointly produce, and it enters the proofs only through  $\bar{q}_n^\star = \frac{1}{n} \sum_{i \leq n} \tilde{q}_i$  of (5), for which the denominator  $\bar{q}_n$  of (6) is an  $m$ -component stand-in. Let  $\mathcal{Q}$  be the set of positive functions of the form (6) whose components satisfy Assumption 2; nothing below requires an element of  $\mathcal{Q}$  to integrate to one. Elements of  $\mathcal{Q}$  are parameterized by  $p_q = (\text{means, component weights, covariances, } \alpha)$  ranging over a compact set  $P_q \subset \mathbb{R}^D$ ,  $D = m(kd + k + d(d+1)/2) + m$ .

**Lemma 6** (bounded weights). *Under Assumptions 2–3,  $0 \leq W_{i,n} \leq 1/\delta$  for all  $i \leq n$  and all  $n$ , surely, whatever the sampler does.*

*Proof* All components of (6) are non-negative, so  $\bar{q}_n \geq \delta\pi$  pointwise on  $\Theta$ . Hence  $W_{i,n} = \pi(\theta_i)K_{\epsilon_n}(\rho_i)/\bar{q}_n(\theta_i) \leq K_{\epsilon_n}(\rho_i)/\delta \leq 1/\delta$ , the last step by  $K_\epsilon \leq 1$  (Assumption 1). The prior cancels exactly, so the bound needs neither a density-ratio condition nor a moment condition.  $\square$

**Lemma 7** (uniform SLLN for bounded martingale differences). *Let  $P \subset \mathbb{R}^D$  be compact and  $\{\psi_p\}_{p \in P}$  measurable functions on a measurable space  $\mathcal{X}$  with  $\sup_p \|\psi_p\|_\infty \leq B$  and  $|\psi_p(x) - \psi_{p'}(x)| \leq \Lambda \|p - p'\|$  for every  $x \in \mathcal{X}$ . Let  $(\xi_i)$  be  $\mathcal{X}$ -valued with  $\xi_i$   $\mathcal{F}_i$ -measurable. Then*

$$\sup_{p \in P} \left| \frac{1}{n} \sum_{i \leq n} (\psi_p(\xi_i) - \mathbb{E}_{i-1} \psi_p(\xi_i)) \right| \longrightarrow 0 \quad a.s.$$

*Proof* Fix  $p$ . The variables  $D_i^p := \psi_p(\xi_i) - \mathbb{E}_{i-1} \psi_p(\xi_i)$  form a martingale difference sequence with  $|D_i^p| \leq 2B$ , so by the Azuma–Hoeffding inequality  $\mathbb{P}(|\sum_{i \leq n} D_i^p| \geq nt) \leq 2 \exp(-nt^2/8B^2)$  for every  $t > 0$ . Fix  $\eta > 0$  and let  $P_\eta$  be a finite  $\eta$ -net of  $P$ , of cardinality at most  $C_P \eta^{-D}$ . A union bound gives  $\mathbb{P}(\max_{p \in P_\eta} |\sum_{i \leq n} D_i^p| \geq nt) \leq 2C_P \eta^{-D} \exp(-nt^2/8B^2)$ , which is summable in  $n$  (the net is fixed, so its cardinality does not grow with  $n$ ); by Borel–Cantelli,  $\limsup_n \max_{p \in P_\eta} |\frac{1}{n} \sum_{i \leq n} D_i^p| \leq t$  a.s. For arbitrary  $p \in P$  pick  $p' \in P_\eta$  with  $\|p - p'\| \leq \eta$ ; then  $|D_i^p - D_i^{p'}| \leq 2\Lambda\eta$  by the Lipschitz hypothesis (which also bounds the conditional expectations), so  $\limsup_n \sup_{p \in P} |\frac{1}{n} \sum_{i \leq n} D_i^p| \leq t + 2\Lambda\eta$  a.s. Intersecting the almost-sure events over the countable family  $t = 1/r, \eta = 1/j$  with  $r, j \in \mathbb{N}$ , gives the claim.  $\square$

**Lemma 8** (conditional-mean identity). *Let  $\phi : \Theta \rightarrow \mathbb{R}$  be bounded measurable, let  $\ell \in \{1, 2\}$  (an exponent, not the fidelity ratio  $r$ ), and let  $\epsilon \in [a, \epsilon_0]$  for some  $a > 0$  and the positive function  $q \geq \delta\pi$  be either deterministic or  $\mathcal{F}_{i-1}$ -measurable. Then under (B5), writing  $L_\epsilon^{(1)} := L_\epsilon$ ,*

$$\mathbb{E}_{i-1} \left[ \frac{\phi(\theta_i) K_\epsilon(\rho_i)^\ell}{q(\theta_i)^\ell} \right] = \int_\Theta \phi L_\epsilon^{(\ell)} \frac{\tilde{q}_i}{q^\ell} d\theta.$$

*Proof* Condition on  $\mathcal{F}_{i-1}$ , given which  $\epsilon$  and  $q$  are constants, and integrate first over  $\rho_i \sim p(\cdot \mid \theta_i)$ , which gives the factor  $L_\epsilon^{(\ell)}(\theta_i) = \mathbb{E}[K_\epsilon(\rho)^\ell \mid \theta_i]$ , then over  $\theta_i \sim \tilde{q}_i$ . The  $\mathcal{F}_{i-1}$ -measurable case is what Steps 2 and 3 of the proof of Theorem 3 need; an  $\mathcal{F}_\infty$ -measurable  $q$ , such as the limit  $\bar{q}_\infty$ , would *not* be admissible here, which is why that proof centers on predictable proxies rather than on the limit. Fubini applies because the integrand is bounded by  $\|\phi\|_\infty / (\delta\pi_-)^\ell$ . The case  $\ell = 1, \phi = \pi h$  is the one used for first moments;  $\ell = 2, \phi = \pi^2 h^2$  gives the conditional second moments of Step 3(b) and Corollary 4.  $\square$

**The class used below.**

Take  $\mathcal{X} = \Theta \times [0, \infty)$ ,  $\xi_i = (\theta_i, \rho_i)$ , and, for a fixed bounded measurable  $h$ ,

$$\psi_{(\epsilon, p_q)}(\theta, \rho) := \frac{\pi(\theta) h(\theta) K_\epsilon(\rho)}{q_{p_q}(\theta)}, \quad (\epsilon, p_q) \in P_a := [a, \epsilon_0] \times P_q, \quad a \in (0, \epsilon_0]. \quad (\text{B6})$$

Each  $P_a$  is compact and deterministic, which matters: Lemma 7 cannot be applied over a parameter set that depends on the run, and  $[\epsilon_\infty, \epsilon_0]$  would be such a set when  $\epsilon_\infty$  is random. We therefore apply the lemma on  $P_{1/j}$  for every  $j \in \mathbb{N}$  with  $1/j \leq \epsilon_0$

and intersect the countably many almost-sure events; since  $\epsilon_n \downarrow \epsilon_\infty > 0$  a.s., each path has  $\epsilon_n \in [1/j, \epsilon_0]$  for all  $n$  once  $1/j \leq \epsilon_\infty$ , so the resulting uniform bound covers  $(\epsilon_n, \bar{q}_n)$  on that path. No deterministic bandwidth floor is required. On  $P_a$ ,  $\sup \|\psi\|_\infty \leq \|h\|_\infty / \delta$  by the computation of Lemma 6. For the Lipschitz hypothesis: each truncated component density  $\varphi(\theta; \mu_j, \Sigma) / Z(\mu_j, \Sigma)$  has parameter-derivatives bounded uniformly over  $\theta \in \Theta$ , because  $\Theta$  is compact,  $\sigma_-^2 I \preceq \Sigma \preceq \sigma_+^2 I$  bounds  $\varphi$  and its derivatives, and the truncation mass is bounded below,  $Z(\mu_j, \Sigma) \geq c_0 > 0$ , uniformly for  $\mu_j \in \Theta$ . Here it matters that the implementation's constant is the *product of per-axis* in-box masses, not the exact joint mass: each factor is  $\Phi((\text{hi}_d - \mu_{jd})/\sigma_d) - \Phi((\text{lo}_d - \mu_{jd})/\sigma_d)$ , and since  $\mu_{jd}$  lies in  $[\text{lo}_d, \text{hi}_d]$  that interval contains a half-width  $w_d/2$  on one side of  $\mu_{jd}$ , giving a factor at least  $\Phi(w_d/2\sigma_+) - \frac{1}{2} > 0$ ; the product over coordinates,  $\prod_d (\Phi(w_d/2\sigma_+) - \frac{1}{2})$ , is therefore bounded below by a constant depending only on the box and  $\sigma_+$ , and is smooth in  $(\mu_j, \Sigma)$  with bounded derivatives there. This is where Assumption 2's box hypothesis is used; compactness of  $\Theta$  alone would not give it. Hence every  $q \in \mathcal{Q}$  (a set of positive *functions*, not necessarily normalized; Assumption 2) obeys  $\delta\pi_- \leq q \leq q_+ < \infty$  and  $q \mapsto q(\theta)$  is Lipschitz in  $p_q$  uniformly in  $\theta$ ; so is  $1/q$ , since  $q$  is bounded away from 0. With  $K_\epsilon$  Lipschitz in  $\epsilon$  (Assumption 1) and  $\pi h$  bounded, (B6) satisfies the hypotheses of Lemma 7. Note that on  $\{1/j \leq \epsilon_\infty\}$  we have  $(\epsilon_n, \bar{q}_n) \in P_{1/j}$  for every  $n$ , and that  $\bar{q}_\infty \geq \delta\pi$  by taking uniform limits.

## B.2 Proof of Theorem 1

Work on the event  $\{1/j \leq \epsilon_\infty\}$  for a fixed  $j$ , so that  $\hat{p}_n := (\epsilon_n, \bar{q}_n) \in P_{1/j}$  for every  $n$ ; the conclusion then follows on the union over  $j$ , which has probability one. Fix a bounded measurable  $h$  and define, for each *fixed*  $p \in P_{1/j}$ ,  $\Gamma_i(p) := \mathbb{E}_{i-1}[\psi_p(\xi_i)]$  and  $\bar{\Gamma}_n(p) := \frac{1}{n} \sum_{i \leq n} \Gamma_i(p)$ . On the almost-sure event of Lemma 7 for  $P_{1/j}$  the bound holds simultaneously for all  $p \in P_{1/j}$ , so the *function*  $\bar{\Gamma}_n$  may be evaluated at the random  $\hat{p}_n$ :

$$\frac{1}{n} \sum_{i \leq n} W_{i,n} h(\theta_i) = \bar{\Gamma}_n(\hat{p}_n) + o(1) \stackrel{\text{Lem. 8}}{=} \int_{\Theta} \pi h L_{\epsilon_n} \frac{\bar{q}_n^*}{\bar{q}_n} d\theta + o(1) \quad \text{a.s.}, \quad (\text{B7})$$

using  $\frac{1}{n} \sum_{i \leq n} \tilde{q}_i = \bar{q}_n^*$ . The middle term is the function  $\bar{\Gamma}_n(\cdot)$  evaluated at  $\hat{p}_n$  and must not be read as  $\mathbb{E}_{i-1}[\psi_{\hat{p}_n}(\xi_i)]$ :  $\hat{p}_n$  depends on draws after  $i$ , is not  $\mathcal{F}_{i-1}$ -measurable, and that conditional expectation is not what Lemma 7 controls. Split the integral against  $\int \pi h L_{\epsilon_\infty}$ :

$$\left| \int \pi h L_{\epsilon_n} \frac{\bar{q}_n^*}{\bar{q}_n} - \int \pi h L_{\epsilon_\infty} r \right| \leq \|h\|_\infty \pi_+ |\Theta| \left[ R L_K(1/j) |\epsilon_n - \epsilon_\infty| + \left\| \frac{\bar{q}_n^*}{\bar{q}_n} - r \right\|_\infty \right],$$

on the event  $\{1/j \leq \epsilon_\infty\}$  of the localization above, using  $|L_{\epsilon_n} - L_{\epsilon_\infty}| \leq L_K(1/j) |\epsilon_n - \epsilon_\infty|$  (Assumption 1 and Jensen; every  $\epsilon_n$  and  $\epsilon_\infty$  lies in  $[1/j, \epsilon_0]$  there) and  $L_{\epsilon_\infty} \leq 1$ , and where  $R < \infty$  bounds  $\bar{q}_n^*/\bar{q}_n$  for all large  $n$ . That bound needs no structural bound on  $\tilde{q}_i$ , of which we assume none: (8) sends the ratio uniformly to  $r \leq r_+$ . Both bracketed terms vanish, the first because  $\epsilon_n \rightarrow \epsilon_\infty$  and the second by (8). Hence  $\frac{1}{n} \sum_i W_{i,n} h(\theta_i) \rightarrow \int \pi h L_{\epsilon_\infty} r$  a.s. Applying this with  $h = f$  and with  $h \equiv 1$  and

dividing, which is legitimate because  $Z_r = \int \pi L_{\epsilon_\infty} r \geq r_- Z_{\epsilon_\infty} > 0$ , gives  $\hat{\pi}_n(f) \rightarrow \pi_{\epsilon_\infty}^r(f)$  a.s. Corollary 2 is the case  $r \equiv 1$ .  $\square$

**Remark 1.** Consistency used Assumption 5(i) only through (8) and  $\epsilon_n \rightarrow \epsilon_\infty$ , and nothing at all about the form of  $\tilde{q}_i$ . This is what makes the theorem a statement about the algorithm as implemented rather than about an idealized variant of it: any sampler whose draw mixture  $\bar{q}_n$  tracks, in the ratio sense of (8), is covered, the archive rule, the capped rejection and the underflow redraws included.

### B.3 Proof of Theorem 3 and Corollary 4

Let  $g := f - \pi_{\epsilon_\infty}^r(f)$ , a fixed bounded function satisfying  $\int \pi g L_{\epsilon_\infty} r = 0$  (Assumption 5(ii) makes  $\epsilon_\infty$  and  $r$ , hence  $\pi_{\epsilon_\infty}^r(f)$ , deterministic). Since  $\sum_i W_{i,n} g(\theta_i) = (\sum_i W_{i,n})(\hat{\pi}_n(f) - \pi_{\epsilon_\infty}^r(f))$  identically and  $\frac{1}{n} \sum_i W_{i,n} \rightarrow Z_r > 0$  a.s. (Theorem 1 with  $h \equiv 1$ ), by Slutsky's theorem (which preserves stable convergence) it suffices to prove

$$S_n := \frac{1}{\sqrt{n}} \sum_{i \leq n} \frac{\pi(\theta_i) g(\theta_i) K_{\epsilon_n}(\rho_i)}{\bar{q}_n(\theta_i)} \longrightarrow \mathcal{N}(0, v^2) \text{ stably}, \quad v^2 := \int_{\Theta} \pi^2 g^2 L_{\epsilon_\infty}^{(2)} \frac{\tilde{q}_\infty}{\bar{q}_\infty^2} d\theta, \quad (\text{B8})$$

after which  $\sigma_f^2 = v^2 / Z_r^2$ , which is (A3).

*Step 1 (replacing the parameters by predictable proxies).*  $\bar{q}_\infty$  and  $\epsilon_\infty$  are in general only  $\mathcal{F}_\infty$ -measurable, so an array built from them is *not* adapted and Lemma 8 does not apply to it. We therefore center on the predictable pair instead: let  $\bar{q}_{i-1} \in \mathcal{Q}$  be the denominator (6) formed from the first  $i-1$  draws and  $\epsilon_{i-1}$  the bandwidth in force at the  $i$ -th proposal, both  $\mathcal{F}_{i-1}$ -measurable, and put

$$V_i := \frac{\pi(\theta_i) g(\theta_i) K_{\epsilon_{i-1}}(\rho_i)}{\bar{q}_{i-1}(\theta_i)}.$$

Since  $\bar{q}_n, \bar{q}_{i-1} \geq \delta \pi_-$  and  $K \leq 1$ , and  $|\epsilon_n - \epsilon_{i-1}| \leq |\epsilon_n - \epsilon_\infty| + |\epsilon_{i-1} - \epsilon_\infty|$  (likewise for  $\bar{q}$ ),

$$\left| S_n - \frac{1}{\sqrt{n}} \sum_{i \leq n} V_i \right| \leq C \sqrt{n} (|\epsilon_n - \epsilon_\infty| + \|\bar{q}_n - \bar{q}_\infty\|_\infty) + \frac{C}{\sqrt{n}} \sum_{i \leq n} (|\epsilon_{i-1} - \epsilon_\infty| + \|\bar{q}_{i-1} - \bar{q}_\infty\|_\infty)$$

with  $C = \|g\|_\infty \pi_+ \max\{L_K(\epsilon_\infty)/(\delta \pi_-), (\delta \pi_-)^{-2}\}$ . This is a single constant, since Assumption 5(ii) makes  $\epsilon_\infty$  deterministic, so the central limit theorem needs no localization; both brackets vanish by (A1). This step is what the rates buy; the remainder of the proof never touches  $\bar{q}_n$  or  $\epsilon_n$  again.

*Step 2 (drift).* Lemma 8 with  $\ell = 1$ ,  $\phi = \pi g$  applies to  $V_i$  because  $(\epsilon_{i-1}, \bar{q}_{i-1})$  is  $\mathcal{F}_{i-1}$ -measurable and  $g$  is a fixed function under the central-limit clause of Assumption 5; with  $\int \pi g L_{\epsilon_\infty} r = 0$ ,

$$c_i := \mathbb{E}_{i-1} V_i = \int \pi g \left( L_{\epsilon_{i-1}} \frac{\tilde{q}_i}{\bar{q}_{i-1}} - L_{\epsilon_\infty} r \right) d\theta,$$

$$|c_i| \leq C'(|\epsilon_{i-1} - \epsilon_\infty| + \|\tilde{q}_i - \tilde{q}_\infty\|_\infty + \|\bar{q}_{i-1} - \bar{q}_\infty\|_\infty),$$

$C'$  depending only on  $\|g\|_\infty, \pi_\pm, \delta, |\Theta|, L_K(\epsilon_\infty)$ . The second half of (A1) is exactly the statement that  $n^{-1/2} \sum_{i \leq n}$  of that bound tends to zero, so  $n^{-1/2} \sum_{i \leq n} c_i \rightarrow 0$ . (Under the sufficient condition that each of the three terms is  $O(i^{-1/2-a})$  this holds in each of the cases  $a < 1/2$ ,  $a = 1/2$ ,  $a > 1/2$ , where the partial sums are  $O(n^{1/2-a})$ ,  $O(\log n)$  and  $O(1)$  respectively.)

*Step 3 (martingale CLT).* Let  $X_{n,i} := n^{-1/2}(V_i - c_i)$ , a martingale difference array with respect to the filtration  $(\mathcal{F}_i)$ , which does not depend on  $n$  and is therefore nested. Two conditions: (a) *Conditional Lindeberg.*  $|X_{n,i}| \leq 2\|g\|_\infty \pi_+ / (\delta \pi_- \sqrt{n})$  surely, so for every  $\eta > 0$  the truncated sum  $\sum_i \mathbb{E}_{i-1}[X_{n,i}^2 \mathbf{1}\{|X_{n,i}| > \eta\}]$  is exactly zero for all  $n$  large. The defensive floor makes this step trivial where an unbounded-weight scheme would need a uniform-integrability argument. (b) *Conditional variance.* Lemma 8 with  $\ell = 2$ ,  $\phi = \pi^2 g^2$  gives

$$\mathbb{E}_{i-1} V_i^2 = \int \frac{\pi^2 g^2 L_{\epsilon_{i-1}}^{(2)}}{\bar{q}_{i-1}^2} \tilde{q}_i d\theta = v^2 + R_i,$$

$$v^2 = \int \pi^2 g^2 L_{\epsilon_\infty}^{(2)} \frac{\tilde{q}_\infty}{\bar{q}_\infty^2} d\theta, \quad |R_i| \leq C''(|\epsilon_{i-1} - \epsilon_\infty| + \|\tilde{q}_i - \tilde{q}_\infty\|_\infty + \|\bar{q}_{i-1} - \bar{q}_\infty\|_\infty),$$

where  $C''$  carries  $2L_K(\epsilon_\infty)$  for the squared kernel, since  $|K_\epsilon^2 - K_{\epsilon'}^2| = |K_\epsilon - K_{\epsilon'}|(K_\epsilon + K_{\epsilon'}) \leq 2L_K(\epsilon_\infty)|\epsilon - \epsilon'|$ . The bound on  $|R_i|$  has vanishing Cesàro average, a fortiori from (A1), so  $\frac{1}{n} \sum_{i \leq n} \mathbb{E}_{i-1} V_i^2 \rightarrow v^2$  a.s.; with  $\frac{1}{n} \sum_i c_i^2 \rightarrow 0$  from Step 2,  $\sum_i \mathbb{E}_{i-1} X_{n,i}^2 = \frac{1}{n} \sum_i (\mathbb{E}_{i-1} V_i^2 - c_i^2) \rightarrow v^2$  a.s. Note that  $v^2$  is permitted to be random here: an  $\mathcal{F}_\infty$ -measurable limit of the conditional variances is the setting the martingale CLT below covers, and predictability was needed only inside  $\mathbb{E}_{i-1}$ . The martingale central limit theorem for arrays with nested  $\sigma$ -fields, in its conditional-variance form [31, Cor. 3.1, pp. 58–59], then gives  $\sum_i X_{n,i} \rightarrow \mathcal{N}(0, v^2)$  stably; the convergence is stable rather than merely distributional because  $v^2$  is  $\mathcal{F}_\infty$ -measurable and possibly random. Combining Steps 1–3 gives (B8).  $\square$

*Proof of Corollary 4* Write  $\hat{\sigma}_n^2 = [\frac{1}{n} \sum_i W_{i,n}^2 (f(\theta_i) - \hat{\pi}_n(f))^2] / [\frac{1}{n} \sum_i W_{i,n}]^2$ . In the numerator,  $(f - \hat{\pi}_n(f))^2 = g^2 + 2g(\pi_{\epsilon_\infty}^r(f) - \hat{\pi}_n(f)) + (\pi_{\epsilon_\infty}^r(f) - \hat{\pi}_n(f))^2$ , and  $W_{i,n} \leq 1/\delta$  with  $\hat{\pi}_n(f) \rightarrow \pi_{\epsilon_\infty}^r(f)$  a.s. (Theorem 1), so the cross terms vanish and it suffices to treat  $\frac{1}{n} \sum_i W_{i,n}^2 g(\theta_i)^2$ . Apply Lemma 7 on  $P_a$  with  $a = \epsilon_\infty$  (deterministic here, so no localization is needed) to the class  $\psi_{(\epsilon, p_q)}(\theta, \rho) = \pi(\theta)^2 g(\theta)^2 K_\epsilon(\rho)^2 / q_{p_q}(\theta)^2$ , which is bounded by  $\|g\|_\infty^2 / \delta^2$  and Lipschitz in  $(\epsilon, p_q)$  by the argument of (B6) with  $2L_K(\epsilon_\infty)$  in place of  $L_K$ ; the substitution of  $\hat{p}_n$  used in (B7), followed by Lemma 8 with  $\ell = 2$ ,  $\phi = \pi^2 g^2$  (a fixed function under Assumption 5(ii)), gives  $\frac{1}{n} \sum_i W_{i,n}^2 g^2(\theta_i) = \int \pi^2 g^2 L_{\epsilon_n}^{(2)} (\bar{q}_n^* / \bar{q}_n) \bar{q}_n^{-1} + o(1)$ , and  $\epsilon_n \rightarrow \epsilon_\infty$ ,  $\|\bar{q}_n^* / \bar{q}_n - r\|_\infty \rightarrow 0$  and  $\|\bar{q}_n - \bar{q}_\infty\|_\infty \rightarrow 0$  (the last from (A1)) send this to  $\int \pi^2 g^2 L_{\epsilon_\infty}^{(2)} r / \bar{q}_\infty = \int \pi^2 g^2 L_{\epsilon_\infty}^{(2)} \tilde{q}_\infty / \bar{q}_\infty^2 = v^2$  a.s., using  $r = \tilde{q}_\infty / \bar{q}_\infty$ . The denominator tends to  $Z_r^2$ , so  $\hat{\sigma}_n^2 \rightarrow v^2 / Z_r^2 = \sigma_f^2$  a.s. Because the convergence in Theorem 3 is stable, the pair  $(\sqrt{n}(\hat{\pi}_n(f) - \pi_{\epsilon_\infty}^r(f)), \hat{\sigma}_n)$  converges jointly to  $(\sigma_f \mathcal{Z}, \sigma_f)$ , because stable convergence is equivalent to joint convergence with every  $\mathcal{F}_\infty$ -measurable variable and  $\sigma_f$  is one. With  $\sigma_f > 0$  a.s. the continuous mapping theorem then gives  $\mathcal{Z} \sim \mathcal{N}(0, 1)$  as the limit of the ratio. Marginal convergence to a mixed normal alone would not suffice.  $\square$

**Remark 2** (growing snapshot count). Nothing above requires  $m$  to be fixed except the compactness and fixed dimension  $D$  of  $P_q$ , and the fixed floor  $\delta$ . The implementation ties  $\delta$  to  $m$  as  $0.5/(m+1)$ , so the two must be decoupled before  $m$  is allowed to grow, since Lemma 6’s bound is  $1/\delta$ . With  $\delta$  fixed and  $m = m_n \rightarrow \infty$ , run the proof of Lemma 7 with an  $n$ -dependent net: the union bound costs  $D_n \log(1/\eta_n)$  with  $D_n = \Theta(m_n)$ , and the Lipschitz constant over the mixture weights grows at most polynomially in  $m_n$ , so choosing  $\eta_n \rightarrow 0$  fast enough leaves the Borel–Cantelli step summable for any  $m_n = o(n/\log n)$ . In that regime the snapshot clause of Assumption 5 follows from  $V_n/m_n \rightarrow 0$ , with  $V_n = \sum_{i < n} \|q_{i+1} - q_i\|_\infty$  the total variation of the proposal path, rather than having to be assumed: the draw-proportional  $m_n$ -point mixture is then a converging quadrature of the empirical mixture. The retroactive pass costs  $\mathcal{O}(n m_n k)$ .

## B.4 Proof of Proposition 5

Write  $\zeta_0 := \pi_{\epsilon_\infty}(|r - 1|)$ . Under the hypothesis,  $\zeta_0 \leq \eta_A + \pi_{\epsilon_\infty}(A^c) = \zeta$ . For bounded  $f$ , since  $\pi_{\epsilon_\infty}^r(f) = \pi_{\epsilon_\infty}(fr)/\pi_{\epsilon_\infty}(r)$ ,

$$\pi_{\epsilon_\infty}^r(f) - \pi_{\epsilon_\infty}(f) = \frac{\pi_{\epsilon_\infty}(f(r-1)) - \pi_{\epsilon_\infty}(f)\pi_{\epsilon_\infty}(r-1)}{\pi_{\epsilon_\infty}(r)},$$

whose modulus is at most  $2\|f\|_\infty \zeta_0/(1 - \zeta_0)$  because  $\pi_{\epsilon_\infty}(r) \geq 1 - \zeta_0$ . Taking the supremum over  $\|f\|_\infty \leq 1$  and halving bounds the total variation by  $\zeta_0/(1 - \zeta_0)$ ; as  $x \mapsto x/(1 - x)$  increases on  $[0, 1)$ , this is at most  $\zeta/(1 - \zeta)$  whenever  $\zeta < 1$ , which is (A4). For  $\zeta \geq 1$  the bound degenerates and only the trivial  $\|\cdot\|_{\text{TV}} \leq 1$  survives.

The closed form quoted in §A.1(a) for the prior floor’s contribution follows by evaluating (8) for that contribution alone: with  $u = q_\infty/\pi$  and a limiting bootstrap share  $\nu_\infty < \delta$ ,

$$r_{\text{floor}} - 1 = \frac{\nu_\infty + (1 - \nu_\infty)u}{\delta + (1 - \delta)u} - 1 = \frac{(\delta - \nu_\infty)(u - 1)}{\delta + (1 - \delta)u},$$

so on  $\{u \geq c\}$ , splitting at  $u = 1$  and bounding each branch,  $|r_{\text{floor}} - 1| \leq (\delta - \nu_\infty) \max\{(1 - \delta)^{-1}, (\delta + (1 - \delta)c)^{-1}\}$ , which is the  $\eta_A$  of Proposition 5 for  $A = \{u \geq c\}$ . It is identically 1 when  $\nu_\infty \geq \delta$ , and  $\rightarrow 1$  as  $\delta \rightarrow 0$ .  $\square$

## B.5 Status of each assumption in the implementation

## Appendix C Implementation Details

### *Relation to classical ABC-SMC.*

Table C2 sets the method against the generation-staged ABC-SMC/PMC implementation we compare with (the pyABC baseline), property by property. None of the listed properties is intrinsic to ABC-SMC as a family (smooth kernels and richer proposals exist), but each characterizes pyABC as configured. The decisive row is the last: because all state is a pure function of the history, there is nothing to synchronize

between workers, which is what fits the method to the asynchronous island model of §5.2.

### ***Tolerance schedulers.***

Every scheduler is a pure function of the evaluated history and is *monotone by construction*: the propagator clips any proposed bandwidth above the history-reconstructed running minimum  $\epsilon_{\text{hist}}$  (§3.2), so the effective tolerance can only decrease. Let  $\mu$  denote the extra-individual margin (default  $\mu = k$ ); it is unrelated to the snapshot count  $m$  of §4. The three base rules act on the individuals *within* the current bandwidth, i.e. the hard-threshold set  $\{i : \rho_i < \epsilon\}$  (this count drives only the tolerance schedule; the proposal mixture and the posterior estimate use the smooth-kernel weight  $K_\epsilon(\rho_i)$ , to which every evaluated particle contributes):

- **Quantile.**  $\epsilon \leftarrow$  the lower-rank (non-interpolated)  $p$ -th percentile of the accepted losses, default  $p = 50$ ; applied only once at least  $k + m$  individuals are accepted.
- **Geometric decay.**  $\epsilon \leftarrow \epsilon_0 \gamma^e$  after each completed epoch  $e$  of  $k + \mu$  accepted individuals, default  $\gamma = 0.9$ .
- **Acceptance rate.** Over a sliding window of the most recent  $k + \mu$  individuals, if the acceptance rate exceeds  $r_{\text{hi}}$  (default 0.3) tighten by a factor 0.9, otherwise hold; the rule never expands  $\epsilon$  (the low-rate guard  $r_{\text{lo}} = 0.1$  simply holds).

With a smooth (Gaussian or Epanechnikov) kernel the schedulers run in a *kernel-aware* mode: instead of the hard  $\rho < \epsilon$  count,  $\epsilon$  is chosen by bisection so that the per-step kernel-weighted effective-sample-size retention ratio meets a target (default 0.95), which keeps the bandwidth schedule comparable across kernel choices. That search runs on a rank’s first call, then once per `bisect_interval` ( $= k$ ) of its own calls, and again whenever the rank’s cached history is rebuilt after an out-of-order arrival; asynchronous arrivals cause such rebuilds and lockstepped ones do not, which is why the barrierized twin’s bandwidth tightens less often than the asynchronous sampler’s at equal call counts (§6.1).

### ***Proposal covariance.***

The archive  $A_n$  is the top- $k$  history members by smallest discrepancy. Component weights are formed in log-space as  $\bar{W}_j \propto w_j K_{\epsilon_n}(\rho_j)$  and normalized (a uniform fallback is used if all smooth weights underflow). The perturbation covariance is  $\Sigma_n = s [\widehat{\text{Cov}}_{\bar{W}}(A_n) + \lambda_n I]$  with  $s$  the `perturbation_scale` and a two-part jitter  $\lambda_n = 10^{-9} \text{tr } \widehat{\text{Cov}}_{\bar{W}}(A_n)/d + 10^{-12} \bar{b}^2$ , where  $\bar{b}^2$  is the mean squared prior-box width; it is Cholesky-factored once per call. The first term is scale-aware, so a tightly converged archive (marginal standard deviation  $\sim 10^{-4}$ ) is not swamped by the floor as a fixed  $10^{-6}$  would swamp it; the second is a run-independent absolute floor that keeps the matrix factorable for a degenerate archive and supplies the uniform lower spectral bound Assumption 2 needs (Table B1). Each mixture component is truncated to the prior box, with its log normalizing mass computed as the product of the per-axis in-box masses. That is exact for a diagonal covariance, in which case the proposal integrates to one on the support; for a correlated covariance it is an approximation, so the evaluated proposal is a positive function that need not integrate to one. This is why

§4 states its denominator assumption for positive functions and books the discrepancy as factor (c) of  $r$  (§A.1).

***Matched pyABC acceptor.***

For pyABC, pyABC’s `UniformAcceptor` is replaced, for smooth kernels, by a probabilistic-rejection acceptor that accepts a candidate of discrepancy  $\rho$  with probability  $K_\epsilon(\rho)$  (peak-normalized so  $K_\epsilon(0) = 1$ ), using the *same* kernel  $K_\epsilon$  as the asynchronous sampler; the hard kernel falls back to `UniformAcceptor`. The rejection step of each replicate draws from an independently seeded generator. This is the sense in which the kernel and acceptor are matched across methods (§5.2).

***Bounded-memory posterior estimator.***

The retroactive AMIS estimator (7) evaluates the snapshot denominator  $\bar{q}_n$ , a draw-proportional mixture of  $m \leq S+1 = 21$  history-reconstructed proposals with a defensive prior component (mass floored at  $0.5/(m+1)$ ), at each of the  $n$  evaluated particles. The total cost is  $\mathcal{O}(nmk) = \mathcal{O}(nk)$  with  $m$  fixed. The snapshots are taken draw-proportionally from the reconstructed archive (indices  $0, \text{step}, 2\text{step}, \dots$  with  $\text{step} = \max(1, \lfloor |\text{archive}|/S \rfloor)$ ), plus the most recent archive-phase proposal if it is not already among them; hence  $m \leq S+1 = 21$  rather than  $S$ . For near-instantaneous simulators the history reaches  $\mathcal{O}(10^7)$  particles, so the pass is accumulated in chunks of  $2^{16} = 65,536$  history points: peak memory is  $\mathcal{O}(\text{chunk} \cdot m \cdot k)$  rather than  $\mathcal{O}(n)$ . Each particle’s denominator is evaluated independently within a single chunk (there is no log-sum-exp reduction *across* chunk boundaries), so chunking changes memory, not the value, and the result is bit-identical to the unchunked computation. Benchmarked at  $k = 100$ ,  $m = 20$  over  $n$  from  $2.5 \times 10^4$  to  $10^6$ , the pass costs 49.3 s per  $10^6$  particles on a single core and is linear in  $n$ , so the  $3 \times 10^7$ -particle Gaussian history extrapolates to  $\approx 25$  minutes; peak memory is flat in  $n$ , as the chunking predicts. The pass runs after the timed budget and is excluded from the throughput measurements; for a near-instantaneous simulator it is comparable to the timed budget itself, for cost-bearing simulators negligible against the simulation time.

## Appendix D Additional Experimental Protocol

***Effective simulating ranks.***

A “ $W$ -worker” point allocates  $W$  physical cores to both samplers. Every rank of the asynchronous sampler evaluates; pyABC’s MPI sampler is root-driven, so rank 0 dispatches without simulating and pyABC has  $W-1$  simulating ranks. We report the allocated  $W$  for both, which flattens the asynchronous sampler by  $1/W$  (2.1% at  $W=48$ , 0.3% at 384) and is not corrected.

***Seeding.***

Each experiment derives its replicate seeds deterministically from a single base seed via `make_seeds`: a `random.Random(base)` stream emits unique non-negative 31-bit integers, one per replicate, portable across platforms and independent of any global RNG

state. Derived seeds (observed-data generation, the pyABC rejection step) use a structured BLAKE2b hash (`stable_seed`) of their inputs, so identical inputs reproduce identical seeds across fresh and resumed (`--extend`) runs.

***Crash recovery (kill-and-resume).***

On the Gaussian-mean benchmark (4 workers, a  $1.2 \times 10^5$ -simulation budget) a run hard-killed partway through inference and relaunched with an identical command reloads the periodic population checkpoint, completes the same total simulation budget with no duplicated records, and recovers the analytic posterior mean (absolute error  $7.6 \times 10^{-3}$ , against  $7.5 \times 10^{-4}$  for an uninterrupted run). It is not a bit-identical replay (the two posterior means differ by  $6.9 \times 10^{-3}$ ), because the candidate generator is re-seeded rather than checkpointed, MPI arrival order is not deterministic, and the online AMIS snapshot buffer is rebuilt empty on restart. The post-restart proposal sequence therefore follows a different, equally valid adaptation trajectory. This is the distinction §3.1 draws: recovery yields a valid posterior over the actual evaluated history, not the identical one.

***Cluster and wall-clock control.***

Experiments run on JUWELS (Forschungszentrum Jülich) under ParaStation MPI, 48 cores per node, scheduled with SLURM. All wall-clock-budgeted methods stop at a fixed wall-clock budget enforced uniformly by a monotonic-clock deadline with first-rank-hit semantics: once any rank trips the deadline it serializes its current state into the records and returns, so a partial final generation is recorded rather than discarded (the simulation-limited studies marked † in Table D3, the SBC and kill-and-resume runs, instead stop at a fixed per-trial simulation count). Posterior estimation and plotting run off the timed inference path, so they do not count against the budget. Matched-budget tolerances (§5.3) count only records of completed simulations; pyABC also writes one record per accepted particle, and mixing the two would flatter it.

***Benchmark configurations.***

Table D3 lists the resolved settings ( $k$  is the archive/population size; the budget is per method run). The strong-scaling studies sweep the worker count from 1 to 288 (Lotka–Volterra; sub-node 1/4/16 and fully-packed multiples of 48 up to six nodes) and to 384 (Cellular Potts; up to eight nodes), with archive size  $k \in \{100, 1000\}$ ; all other studies fix the workers shown.

***Benchmark-specific notes.***

Each benchmark fixes one observed dataset (generated from seed 42) and infers parameters under uniform priors; the discrepancy is a distance between simulated and observed summary statistics. *Gaussian mean*:  $n_{\text{obs}} = 100$  observations  $\sim \mathcal{N}(0, 1)$ , prior  $\mu \sim \mathcal{U}[-5, 5]$ , summary the sample mean, discrepancy  $|\bar{x}_{\text{sim}} - \bar{x}_{\text{obs}}|$ ; the closed-form Gaussian posterior is the calibration and recovery reference. *g-and-k*: the quantile-defined g-and-k distribution ( $c = 0.8$ ),  $n_{\text{obs}} = 1000$ , true  $(A, B, g, k) = (3, 1, 2, 0.5)$ , priors  $A \sim \mathcal{U}[0, 6]$ ,  $B \sim \mathcal{U}[0.1, 4]$ ,  $g \sim \mathcal{U}[0, 5]$ ,  $k \sim \mathcal{U}[0, 1]$ ; summary

the seven octiles (the 0.125, ..., 0.875 quantiles), discrepancy Euclidean. *Lotka–Volterra*: a Gillespie stochastic simulation [32] of the four-reaction predator–prey system from  $(X_0, Y_0) = (50, 100)$  over  $T = 30$ , true rates  $\theta = (0.5, 0.025, 0.025, 0.5)$ , priors  $\theta_1, \theta_4 \sim \mathcal{U}[0.05, 2]$  and  $\theta_2, \theta_3 \sim \mathcal{U}[0.001, 0.2]$ ; summary the six statistics  $(\bar{X}, \text{sd } X, \bar{Y}, \text{sd } Y, \log(1 + \text{var } X/\bar{X}), \log(1 + \text{var } Y/\bar{Y}))$ , discrepancy Euclidean. *Cellular Potts*: the real `cellsInSilico` (NASTJA) simulator; the setup and the screening that fixed it are in Appendix E. Its per-evaluation cost dominates, which is why it uses the longest wall-clock budget.

### Computing environment.

All experiments ran on the JUWELS Cluster at the Jülich Supercomputing Centre: dual-socket Intel Xeon Platinum 8168 (Skylake) nodes with 48 physical cores and  $\approx 94$  GB memory per node, connected by Mellanox EDR InfiniBand. The software stack is ParaStationMPI 5.10 (the 2025 GCC toolchain), Python 3.12.3, NumPy 1.26.4, SciPy 1.13.1, mpi4py 4.0.1, and pyABC 0.12.17, with POT [33] for the sliced Wasserstein distance; the asynchronous sampler runs as a propagator in our fork of Propulate. Unless noted, ranks are placed one per physical core, so a “ $W$ -worker” point occupies  $W$  physical cores packed onto  $\lceil W/48 \rceil$  fully-reserved nodes. The production experiment of the three cheap benchmarks (Gaussian mean, g-and-k, Lotka–Volterra) instead placed its 48 ranks as 12 per node on four nodes under whole-node reservation, after the single-node submissions failed, which is why its asynchronous throughput is below the packed single-node scaling run’s on the same benchmark (§6.3). The Cellular Potts production run and the straggler, heterogeneity, twin and scaling experiments are single-node or fully packed. Each strong-scaling point reports five replicates (three to five for the largest Cellular Potts points), and figures show medians with inter-quartile ranges.

## Appendix E The Cellular Potts inference setup

The benchmark runs the `cellsInSilico` Cellular Potts tissue model [26] through the NASTJA framework [34]: a  $50^3$  lattice for 501 steps (an  $80^3$  lattice for 1001 steps in the larger configuration), seeded with a small cluster of cells that divide and move, summarized at the final step. The inference problem was designed on a screening experiment of about 12,000 simulations over space-filling designs of six candidate parameters, scored by the ratio of between-parameter to within-parameter variation of each summary block. Three findings fixed the setup. Of the seven summary blocks the shipped configuration weighted equally, two scalars, the log cell count and the log 95th-percentile radius, carry nearly all of the signal (signal-to-noise 12 and 9 against about 1 for the four ten-dimensional curve blocks), so the discrepancy is taken over those two, equally weighted. One evaluation is the feature-average of four replicate simulations at the same parameters, because a single realization is too noisy for the discrepancy to order nearby parameters. And the two summaries resolve exactly two directions (how many cells there are, and how large the cluster is at a given count), so the inferred parameters are the two that move them independently: the division rate (count) and the target cell volume (radius at fixed count), whose responses are

near-orthogonal ( $|\cos| = 0.07$ ). Every other candidate expresses itself through the count: cell motility, adhesion and surface tension are confounded with the division rate at  $|\cos| \geq 0.8$  at  $50^3$ , and at  $80^3$ , with  $6.4\times$  the cells, the confounding sharpens rather than breaks ( $0.81 \rightarrow 0.98$  for the surface term). The benchmark is therefore two-dimensional as a property of the model, not for want of screening. Priors are log-uniform on  $[0.001, 0.2]$  and  $[200, 1200]$  with truths 0.009 and 500; motility is fixed at 1400. Rejection ABC on 2000 prior draws forecast 91% and 64% contraction for the two parameters before any adaptive run, against the 92% and 81% the asynchronous estimator reports (§6.4).

### ***Two configurations.***

Two configurations of the benchmark appear in this paper. The *production* configuration is the two-parameter setup above and supplies every posterior result, Tables 3 and 5, the  $80^3$  systems comparison, and the theory diagnostics of Appendix A. The *earlier* configuration was used by the  $50^3$  twin and scaling experiments (the  $50^3$  points of Fig. 2c, Table F5, Fig. 3 right, Fig. F6). It inferred the cell division rate and motility on normalized  $[0, 1]$  priors mapping to  $[6 \times 10^{-5}, 0.6]$  and  $[0, 10^4]$  (true normalized values 0.0499 and 0.2), scored one simulation per evaluation (5.1 s at  $50^3$ , in-run coefficient of variation 0.10) by a whitened distance over morphological summary features (radial density and pair-correlation profiles, free-area and structure-factor profiles, a DBSCAN dispersion fraction, and log cell count and radius). The screening above found those two parameters confounded, which is why the production configuration replaced them. The systems measurements depend on the configuration only through its runtime distribution, which the predictor takes from the run itself.

## **Appendix F Additional results**

The studies below support the main text and are reported in full here.

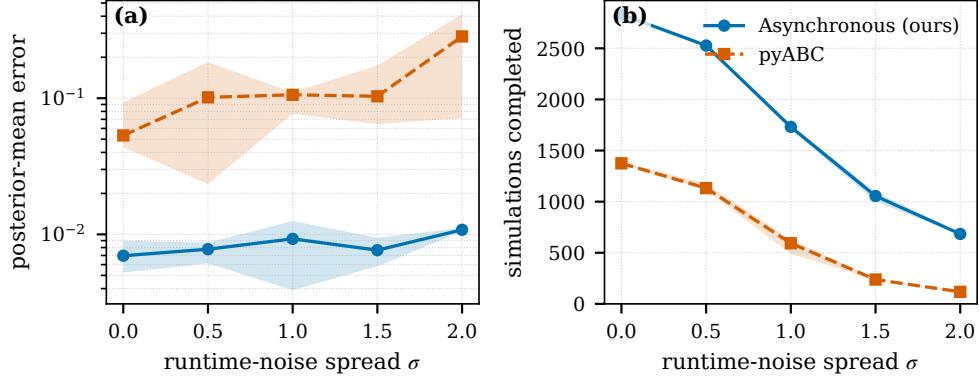
### **F.1 Runtime heterogeneity: archive quality**

#### ***The synthetic instance.***

The heterogeneity study shows the same conversion where the runtime law is known (Fig. F1): at a fixed 60 s budget on the Gaussian benchmark pyABC completes fewer simulations as  $\sigma$  grows and its archive’s posterior-mean error to the analytic posterior climbs from  $\approx 0.05$  to  $\approx 0.28$ , while the asynchronous sampler completes several-fold more and its archive holds  $\approx 0.007$ – $0.011$ . These are archive means; the posterior estimates of both samplers in this experiment are scored in Table F4.

### **F.2 The barrierized twin under runtime heterogeneity and on Cellular Potts**

Table F4 gives the twin’s heterogeneity experiment behind §6.1, with the posteriors of every sampler at the bandwidth its schedule reached and re-reported at the order statistic (§6.4); Table F5 factorizes the twin’s Cellular Potts slowdown into the barrier’s share and the unattributed duration ratio.



**Fig. F1** Inference per unit wall clock under runtime heterogeneity (Gaussian mean, analytic posterior, 48 workers, 60 s budget, five replicates; bands are inter-quartile ranges). Left: error of the archive mean (the unweighted top- $k$  archive, not  $\hat{\pi}_n$ ; §6.4) to the analytic posterior mean, against the runtime spread  $\sigma$ . Right: simulations completed within the budget. The barrier starves pyABC as  $\sigma$  grows; the asynchronous sampler’s error stays flat.

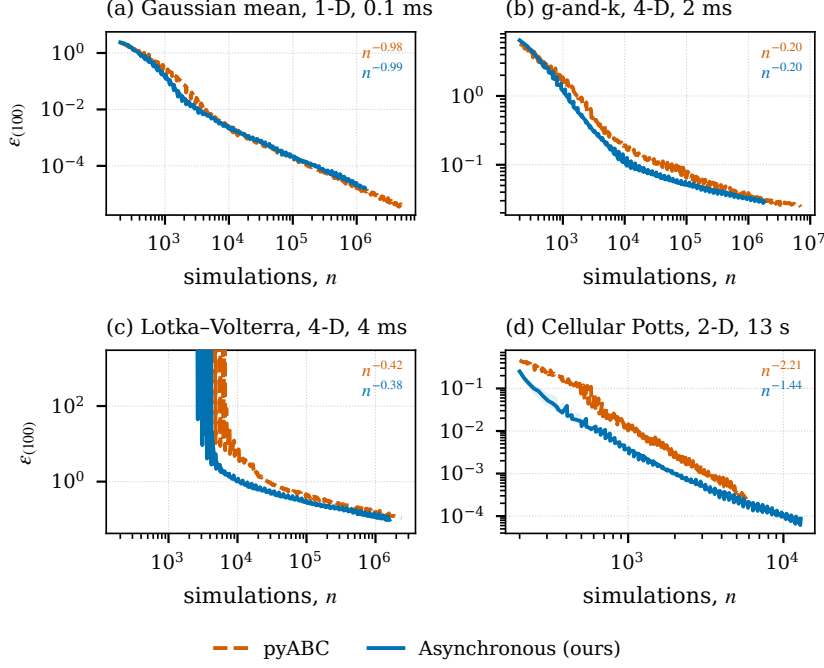
### F.3 Tolerance curves

Figure F2 shows the curves Table 3 is read from.

### F.4 Parameter-coupled runtime

The heterogeneity study above slows simulations independently of their parameters; a sharper test couples runtime *to the inferred parameter*, probing the completion-time selection of §8. On the Gaussian-mean benchmark (48 workers, 60 s budget, five replicates) the post-evaluation delay is  $d(\mu) = \min(d_0 e^{c(\mu-\mu_c)/\ell}, d_{\max})$  with  $d_0 = 0.02$  s,  $\mu_c = 0$ ,  $\ell = 0.15$  and  $d_{\max} = 0.5$  s, swept over  $c \in \{0, 0.5, 1, 2\}$  ( $c = 0$  is uniform runtime), so one region of parameter space evaluates far faster than its mirror image and the every-particle rule over-represents it in the raw archive. The coupling has the intended effect on the sampling rate: as it steepens, aggregate throughput falls by  $\approx 43\%$  for the asynchronous sampler and more steeply for pyABC (Fig. F3a).

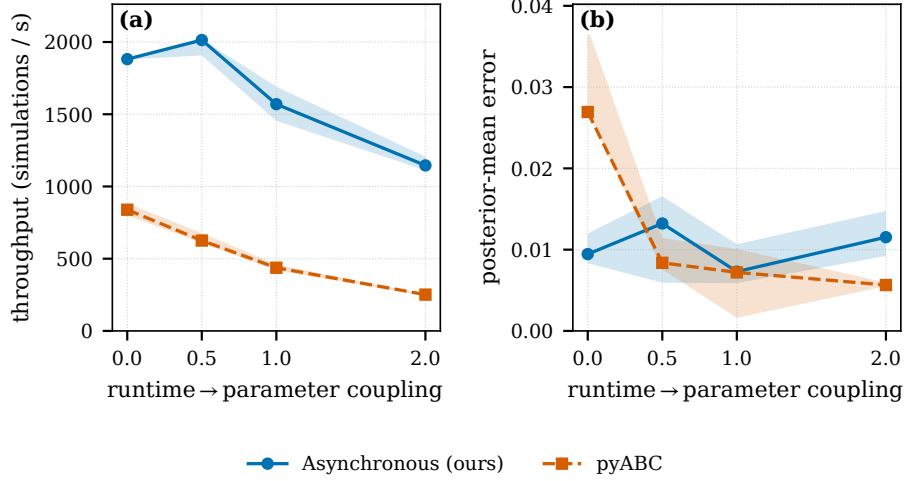
The posterior does not follow. The asynchronous posterior-mean error to the analytic posterior stays  $\approx 0.01$  at every coupling, comparable to pyABC, which discards latecomers (Fig. F3b), with or without AMIS ( $S=0$ ), for the unweighted archive and for the AMIS-reweighted estimator (7), and in a steeper regime (base delay 0.5 s, cap 8 s). The reweighting changes the spread, not the mean: it lowers the effective-sample-size fraction from  $\approx 0.99$  to  $\approx 0.88$  and widens the central 90% interval by  $\approx 15\%$ . A drain-after-deadline variant, which lets every in-flight simulation finish past the deadline, moves the weighted posterior mean by at most 0.013, within the replicate spread and with no systematic direction, so the deadline censoring is negligible at these couplings. The raw archive shows no measurable mean bias at these couplings; AMIS neither creates nor removes one here, and a correction in the manner of anytime Monte Carlo [30] is future work.



**Fig. F2** Tolerance against simulations spent,  $\epsilon_{(100)}(n)$ , asynchronous against pyABC on every benchmark (medians and replicate range over five replicates; the slope over the last decade of  $n$  in the legend). The vertical gap at equal  $n$  is the per-simulation efficiency, the horizontal extent is what the wall clock bought. On Lotka–Volterra 98% of simulations return the extinction fallback, so  $\epsilon_{(100)}$  sits on that plateau until about 5000 draws have produced 100 survivors; the plateau is cut off.

## F.5 Strong scaling on a cheap simulator

Lotka–Volterra is an adversarial simulator for a scaling study. Each evaluation takes  $\approx 4$  ms, so there is little per-evaluation compute to distribute, and an instrumented run (Fig. F4) shows a worker inside the simulator  $\approx 25\%$  of its time at the 48-worker throughput peak, falling to  $\approx 8\%$  and  $\approx 2\%$  as the job spreads from one node to three to six. The mechanism is the method’s single shared population: every completed evaluation is exchanged with all workers so that each can propose from the current archive, an all-to-all exchange whose per-arrival cost grows with the worker count. Aggregate throughput therefore rises almost linearly while the job fits on one node and then declines across fully packed multi-node jobs (Table F6), because adding ranks beyond one node lengthens the exchange faster than it adds useful work. pyABC is given a population equal to the worker count at high core counts (a smaller population cannot occupy every core); it scales steadily and overtakes the asynchronous sampler beyond three nodes. This is the boundary of the asynchronous advantage on a cheap, homogeneous simulator, and it lifts once the simulator carries real per-evaluation cost (Fig. 3, right). The archive size moves the boundary, because the per-arrival proposal reconstruction is  $\mathcal{O}(k)$ . Raising  $k$  to 1000 costs 39% of the single-node peak throughput.



**Fig. F3** Parameter-coupled runtime on the Gaussian-mean benchmark (analytic posterior; 48 workers, 60 s budget, five replicates; bands are inter-quartile ranges). The post-evaluation delay grows with the inferred mean, so one region of parameter space is sampled far more often than its mirror image. (a) Aggregate throughput falls as the coupling steepens (monotonically for pyABC, and from a peak at  $c = 0.5$  for the asynchronous sampler), consistent with the fast region dominating the raw sample counts. (b) Posterior-mean error to the analytic posterior nonetheless does not increase with coupling and stays comparable between the asynchronous sampler and pyABC, which discards latecomers, so the over-representation does not measurably distort the posterior estimate at these coupling levels.

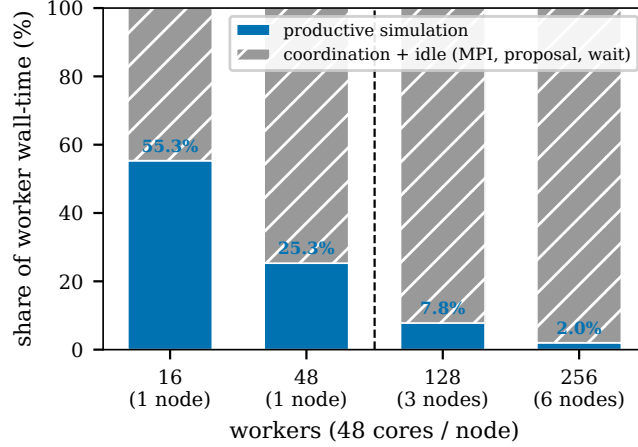
Sweeping  $k$  over  $\{50, 100, 200, 400, 800, 1000\}$  resolves that cost into a smooth monotone decline, with a single step in the crossover: from beyond three nodes to three at  $k \geq 400$  (Table F8).

## F.6 Calibration against the archive size and the snapshot buffer

The calibration tests of §6.4 fix  $k$  and  $S$  per benchmark, which leaves open whether the two efficiency knobs must be retuned as the dimension grows. To test this we sweep them directly on a  $d$ -dimensional Gaussian-mean target,  $y_i \sim \mathcal{N}(\theta, \sigma^2 I_d)$  with  $\theta \in \mathbb{R}^d$  under a flat box prior, whose posterior is known exactly at every  $d$  (SBC against that exact posterior returns 0.496/0.794/0.905/0.952, so the target itself is calibrated and any departure below is a property of the method). The grid is  $d \in \{1, 2, 4, 8, 16\} \times k \in \{50, 100, 200, 400, 800\} \times S \in \{0, 5, 20, 50\}$ , 1000 trials per cell, reporting the same full-history estimator (Table F7).

*The archive size does not, in this range, trade calibration against throughput.*

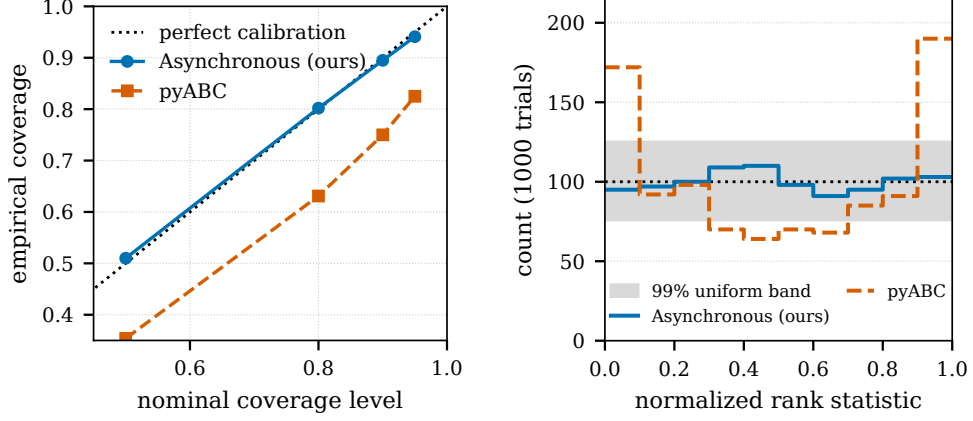
Because  $k$  has both a statistical effect (above) and a systems cost (per-arrival proposal reconstruction is  $\mathcal{O}(k)$ ), so a larger archive is slower at every worker count; §6.3), one might expect to buy calibration with throughput. Over the range we can test, that



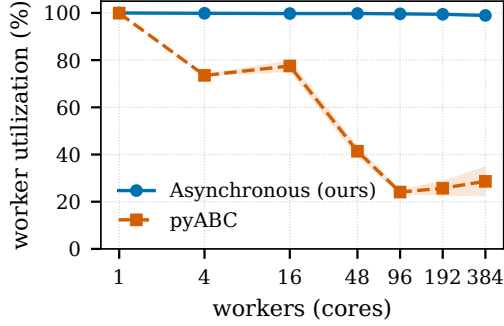
**Fig. F4** Measured decomposition of asynchronous worker wall-time on Lotka–Volterra ( $k = 100$ ; 180s budget) from a dedicated instrumented run. The productive simulation fraction (blue, annotated), the share of worker wall-clock spent inside the simulator, collapses from 55% at 16 workers to 2% at 256, while coordination and idle (MPI exchange, per-arrival proposal reconstruction, serialization, waiting) dominate as the job spreads beyond one node (16/48 = 1 node; 128 = 3 nodes; 256 = 6 nodes). The strong-scaling decline of Fig. 3 is therefore a measured property of coordinating a near-instantaneous simulator.

expectation does not hold. Table F8 puts the throughput cost of  $k$  next to its worst-case miscalibration over the dimensions swept, using a throughput grid matched on everything but  $k$  (Lotka–Volterra, 180s wall cap, five replicates, worker counts 1–288:  $k \in \{100, 1000\}$  from the main scaling run and  $k \in \{50, 200, 400, 800\}$  from a dedicated  $k$ -frontier run sharing that configuration). Of the six archive sizes, only two are Pareto-optimal, and they are the two smallest:  $k = 100$  is simultaneously the best-calibrated setting tested (worst deviation 0.064, at  $d = 4$ ) and within 7% of the fastest (6240 vs 6684 simulations/s at the single-node peak). Every larger archive is dominated by it on *both* axes:  $k = 200$  is 5% slower and nearly three times worse calibrated (0.180),  $k = 400$  is 14% slower and 0.195, and  $k = 800$  costs 31% of throughput and is still slightly worse calibrated than  $k = 100$  (0.074). The only faster archive,  $k = 50$ , is the one that fails calibration (0.250 at  $d = 8$ ). The  $k$  decision is therefore a floor to clear rather than a frontier to negotiate: go far enough above the small-archive failure region and then stop, because further enlargement buys no calibration and costs throughput monotonically.

The two axes come from different benchmarks by design (calibration needs a target whose posterior is known at every  $d$ ; throughput needs the regime where an  $\mathcal{O}(k)$  per-arrival cost is most exposed), so the throughput column is an upper bound on the relative cost of  $k$  and the conclusion is conservative. At scale  $k$  acts as a step in the crossover with pyABC at a population equal to the worker count: beyond three fully packed nodes for  $k \leq 200$ , three nodes for  $k \geq 400$  (last column).



**Fig. F5** Simulation-based calibration on Gaussian-mean inference (1000 trials; the full-history AMIS posterior): empirical coverage versus nominal level (left) and rank histogram (right). The asynchronous estimator is close to calibrated.



**Fig. F6** Cellular Potts  $50^3$  worker utilization in the benchmark’s earlier single-simulation configuration (fraction of worker wall-clock spent inside the simulator) versus worker count (1800 s budget; error bars are  $\pm 1$  s.d.); pyABC uses a population equal to the worker count (Fig. 3, right). The asynchronous sampler stays  $\approx 98.9$ – $99.8\%$  utilized at every scale. pyABC fills every core within a generation but then idles at the barrier waiting for the slowest simulation, so its utilization falls to  $\approx 24$ – $29\%$  ( $41\% \rightarrow 24\% \rightarrow 26\% \rightarrow 29\%$  at 48/96/192/384 workers, i.e.  $\approx 70$ – $76\%$  idle at every multi-node scale). The figure is the measured utilization behind pyABC’s sub-linear scaling in Fig. 3 (right).

## F.7 The point-mass concentration diagnostic

A Wasserstein distance from the unweighted top- $k$  archive to a point mass at the true parameter is floored at the posterior’s own spread (on the Gaussian configuration an exactly correct posterior scores  $0.1\sqrt{2/\pi} = 0.080$ ), and it ranks a collapsed archive above a correct one. We use it only as a concentration diagnostic, for the ablation and sensitivity grids below, and report none of its values as posterior quality.

## F.8 Worker utilization on Cellular Potts at 50<sup>3</sup>

## F.9 Ablation and sensitivity

An ablation (hard kernel; no AMIS) and a hyperparameter grid (perturbation scale, initial-tolerance multiplier, archive size) on the Gaussian-mean benchmark (48 workers, 300s, five replicates), scored by the concentration diagnostic above, find no ingredient removal or grid setting that changes the archive’s concentration (0.07–0.15 throughout, degrading only mildly toward the largest perturbation scales and initial tolerances). The tolerance scheduler is inert under a smooth kernel, since the bandwidth is chosen by ESS-retention bisection regardless of the rule, and was held fixed. Removing AMIS is neutral on this uniform-runtime target, consistent with the parameter-coupled study above. Calibration, which this diagnostic cannot see, is the archive-size sweep above.

## F.10 Lotka–Volterra extinction

The simulator has an absorbing state: if either population reaches zero before  $T_{\max}$  the run returns a fallback discrepancy, which a smooth kernel weights to effectively zero, and at the configured parameters that is the usual outcome (survival 2.6% of 2000 simulations at the true parameter, 1.7% over 400 uniform prior draws). The observed dataset is itself re-drawn until a trajectory survives, so the target is  $p(\theta \mid s_{\text{obs}}, \text{survived})$ , which is internally consistent: the extinct particles’ zero weight is the  $P(\text{survive} \mid \theta)$  factor of the likelihood. Throughput and scaling are unaffected, because an extinct simulation costs what a surviving one costs. Posterior recovery is affected: the informative sample is  $\approx 2\%$  of the nominal budget, so we report Lotka–Volterra as a systems benchmark and treat its posterior curves as diagnostics that bound no method’s attainable accuracy.

## References

- [1] Beaumont, M.A.: Approximate Bayesian computation. *Annual Review of Statistics and Its Application* **6**, 379–403 (2019) <https://doi.org/10.1146/annurev-statistics-030718-105212>
- [2] Sisson, S.A., Fan, Y., Tanaka, M.M.: Sequential Monte Carlo without likelihoods. *Proceedings of the National Academy of Sciences* **104**(6), 1760–1765 (2007) <https://doi.org/10.1073/pnas.0607208104>
- [3] Toni, T., Welch, D., Strelkowa, N., Ipsen, A., Stumpf, M.P.H.: Approximate Bayesian computation scheme for parameter inference and model selection in dynamical systems. *Journal of the Royal Society Interface* **6**(31), 187–202 (2009) <https://doi.org/10.1098/rsif.2008.0172>
- [4] Beaumont, M.A., Cornuet, J.-M., Marin, J.-M., Robert, C.P.: Adaptive approximate Bayesian computation. *Biometrika* **96**(4), 983–990 (2009) <https://doi.org/10.1093/biomet/asp052>

- [5] Del Moral, P., Doucet, A., Jasra, A.: An adaptive sequential Monte Carlo method for approximate Bayesian computation. *Statistics and Computing* **22**(5), 1009–1020 (2012) <https://doi.org/10.1007/s11222-011-9271-y>
- [6] Cornuet, J.-M., Marin, J.-M., Mira, A., Robert, C.P.: Adaptive multiple importance sampling. *Scandinavian Journal of Statistics* **39**(4), 798–812 (2012) <https://doi.org/10.1111/j.1467-9469.2011.00756.x>
- [7] Taubert, O., Weißer, M., Kelle, D., Bayer, M., Knüttel, H., Comito, C., Streit, A., Götz, M.: Massively parallel genetic optimization through asynchronous propagation of populations. In: *International Conference on High Performance Computing (ISC High Performance)*, pp. 106–124 (2023). [https://doi.org/10.1007/978-3-031-32041-5\\_6](https://doi.org/10.1007/978-3-031-32041-5_6)
- [8] Alamoudi, E., Reck, F., Bundgaard, N., Graw, F., Brusch, L., Hasenauer, J., Schälte, Y.: A wall-time minimizing parallelization strategy for approximate Bayesian computation. *PLOS ONE* **19**(2), 0294015 (2024) <https://doi.org/10.1371/journal.pone.0294015>
- [9] Klinger, E., Rickert, D., Hasenauer, J.: pyABC: distributed, likelihood-free inference. *Bioinformatics* **34**(20), 3591–3593 (2018) <https://doi.org/10.1093/bioinformatics/bty361>
- [10] Wilkinson, R.D.: Approximate Bayesian computation (ABC) gives exact results under the assumption of model error. *Statistical Applications in Genetics and Molecular Biology* **12**(2), 129–141 (2013) <https://doi.org/10.1515/sagmb-2013-0010>
- [11] Fearnhead, P., Prangle, D.: Constructing summary statistics for approximate Bayesian computation: semi-automatic approximate Bayesian computation. *Journal of the Royal Statistical Society: Series B* **74**(3), 419–474 (2012) <https://doi.org/10.1111/j.1467-9868.2011.01010.x>
- [12] Drovandi, C.C., Pettitt, A.N.: Estimation of parameters for macroparasite population evolution using approximate Bayesian computation. *Biometrics* **67**(1), 225–233 (2011) <https://doi.org/10.1111/j.1541-0420.2010.01410.x>
- [13] Lenormand, M., Jabot, F., Deffuant, G.: Adaptive approximate Bayesian computation for complex models. *Computational Statistics* **28**(6), 2777–2796 (2013) <https://doi.org/10.1007/s00180-013-0428-3>
- [14] Veach, E., Guibas, L.J.: Optimally combining sampling techniques for Monte Carlo rendering. In: *Proceedings of SIGGRAPH '95*, pp. 419–428 (1995). <https://doi.org/10.1145/218380.218498>
- [15] Cappé, O., Guillin, A., Marin, J.-M., Robert, C.P.: Population Monte Carlo. *Journal of Computational and Graphical Statistics* **13**(4), 907–929 (2004) <https://doi.org/10.1198/106186004000000000>

[//doi.org/10.1198/106186004X12803](https://doi.org/10.1198/106186004X12803)

- [16] Bugallo, M.F., Elvira, V., Martino, L., Luengo, D., Míguez, J., Djurić, P.M.: Adaptive importance sampling: The past, the present, and the future. *IEEE Signal Processing Magazine* **34**(4), 60–79 (2017) <https://doi.org/10.1109/MSP.2017.2699226>
- [17] Marin, J.-M., Pudlo, P., Sedki, M.: Consistency of adaptive importance sampling and recycling schemes. *Bernoulli* **25**(3), 1977–1998 (2019) <https://doi.org/10.3150/18-BEJ1042>
- [18] Vergé, C., Dubarry, C., Del Moral, P., Moulines, E.: On parallel implementation of sequential Monte Carlo methods: The island particle model. *Statistics and Computing* **25**(2), 243–260 (2015) <https://doi.org/10.1007/s11222-013-9429-x>
- [19] Paige, B., Wood, F., Doucet, A., Teh, Y.W.: Asynchronous anytime sequential Monte Carlo. In: *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 27 (2014)
- [20] Murray, L.M., Lee, A., Jacob, P.E.: Parallel resampling in the particle filter. *Journal of Computational and Graphical Statistics* **25**(3), 789–805 (2016) <https://doi.org/10.1080/10618600.2015.1062015>
- [21] Schälte, Y., Klinger, E., Stapor, P., Hasenauer, J.: pyABC 0.12.3: high-performance distributed, likelihood-free inference. *Bioinformatics* **38**(13), 3354–3355 (2022) <https://doi.org/10.1093/bioinformatics/btac321>
- [22] Dutta, R., Schoengens, M., Pacchiardi, L., Ummadisingu, A., Widmer, N., Künzli, P., Onnela, J.-P., Mira, A.: ABCpy: A high-performance computing perspective to approximate Bayesian computation. *Journal of Statistical Software* **100**(7), 1–38 (2021) <https://doi.org/10.18637/jss.v100.i07>
- [23] Lintusaari, J., Vuollekoski, H., Kangasrääsiö, A., Skytén, K., Järvenpää, M., Marttinen, P., Gutmann, M.U., Vehtari, A., Corander, J., Kaski, S.: ELFI: Engine for likelihood-free inference. *Journal of Machine Learning Research* **19**(16), 1–7 (2018)
- [24] Owen, A., Zhou, Y.: Safe and effective importance sampling. *Journal of the American Statistical Association* **95**(449), 135–143 (2000) <https://doi.org/10.1080/01621459.2000.10473909>
- [25] Rayner, G.D., MacGillivray, H.L.: Numerical maximum likelihood estimation for the g-and-k and generalized g-and-h distributions. *Statistics and Computing* **12**(1), 57–75 (2002) <https://doi.org/10.1023/A:1013120305780>
- [26] Berghoff, M., Rosenbauer, J., Hoffmann, F., Schug, A.: Cells in Silico – introducing a high-performance framework for large-scale tissue modeling. *BMC*

- [27] Graner, F., Glazier, J.A.: Simulation of biological cell sorting using a two-dimensional extended Potts model. *Physical Review Letters* **69**(13), 2013–2016 (1992) <https://doi.org/10.1103/PhysRevLett.69.2013>
- [28] Talts, S., Betancourt, M., Simpson, D., Vehtari, A., Gelman, A.: Validating Bayesian inference algorithms with simulation-based calibration. *arXiv preprint arXiv:1804.06788* (2018)
- [29] Bonneel, N., Rabin, J., Peyré, G., Pfister, H.: Sliced and Radon Wasserstein barycenters of measures. *Journal of Mathematical Imaging and Vision* **51**(1), 22–45 (2015) <https://doi.org/10.1007/s10851-014-0506-3>
- [30] Murray, L.M., Singh, S.S., Lee, A.: Anytime Monte Carlo. *Data-Centric Engineering* **2**, 7 (2021) <https://doi.org/10.1017/dce.2021.6>
- [31] Hall, P., Heyde, C.C.: *Martingale Limit Theory and Its Application*. Academic Press, New York (1980)
- [32] Gillespie, D.T.: Exact stochastic simulation of coupled chemical reactions. *The Journal of Physical Chemistry* **81**(25), 2340–2361 (1977) <https://doi.org/10.1021/j100540a008>
- [33] Flamary, R., Courty, N., Gramfort, A., Alaya, M.Z., Boisbunon, A., Chambon, S., Chapel, L., Corenflos, A., Fatras, K., Fournier, N., Gautheron, L., Gayraud, N.T.H., Janati, H., Rakotomamonjy, A., Redko, I., Rolet, A., Schutz, A., Seguy, V., Sutherland, D.J., Tavenard, R., Tong, A., Vayer, T.: POT: Python optimal transport. *Journal of Machine Learning Research* **22**(78), 1–8 (2021)
- [34] Berghoff, M., Kondov, I., Hötzer, J.: Massively parallel stencil code solver with autonomous adaptive block distribution. *IEEE Transactions on Parallel and Distributed Systems* **29**(10), 2282–2296 (2018) <https://doi.org/10.1109/TPDS.2018.2819672>

**Table B1** What the implementation guarantees, what it assumes, and what it violates. “By construction” means the code enforces the property for every run, not that it was checked post hoc.

| Assumption                                                       | Status                                    | Basis                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
|------------------------------------------------------------------|-------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>1</b> kernel, Lipschitz part                                  | by construction                           | Gaussian and Epanechnikov are peak-normalized and Lipschitz in $\epsilon$ on every $[a, \epsilon_0]$ , with $L_K(a) = 2/ea$ and $2/a$ . The hard indicator is not, and is excluded from the theory.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>1</b> $Z_{\epsilon_\infty} > 0$                               | holds for Gaussian                        | The Gaussian kernel is strictly positive, so $L_{\epsilon_\infty} > 0$ and the limiting target is non-degenerate whatever the discrepancy distribution. For a compactly supported kernel (Epanechnikov) it requires the achievable discrepancies to reach inside the support, which we do not verify.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>2</b> denominator class                                       | by construction                           | The prior support is a box and the archive lies in it, so component means do; the covariance carries an additive jitter $10^{-9}\text{tr}(\widehat{\Sigma})/d + 10^{-12}\bar{b}^2$ ( $\bar{b}^2$ the mean squared box width), whose second term is a run-independent floor, giving $\sigma_-^2 > 0$ , and $\sigma_+^2$ follows from the archive lying in a bounded box. The truncation <i>normalizer</i> is the product of per-axis in-box masses, so with a correlated covariance $\bar{q}_n$ does not integrate to one. Assumption <b>2</b> does not require that: it constrains a positive function, and the discrepancy is carried by $r$ instead (§A.1(c)).                                                                                                                                                                                                                                                                                                                                 |
| <b>4</b> adapted filtration                                      | <b>assumed</b>                            | Automatic for a sequential run. Under asynchrony a discrepancy may be in flight when the next candidate is proposed, so the field of realized outcomes cannot serve; the assumption is that a larger filtration exists under which the candidate’s conditional law is still $\bar{q}_i$ . Where completion times steer scheduling, $\bar{q}_i$ is that enlarged-field law rather than the emitted proposal density. Same mechanism as the completion-time selection limitation of §8.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| <b>3</b> prior floor                                             | by construction                           | The retroactive denominator floors the prior component’s mass at $\delta = 0.5/(m+1)$ (Appendix C), so $\bar{q}_n \geq \delta\pi$ and Lemma 6 bounds every posterior weight by $1/\delta$ , independently of what the sampler does.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| <b>5(i)</b> fidelity (8)                                         | assumed; two of its four factors measured | The one place the sampler enters for Theorem 1. Its four factors are decomposed in §A.1; two of them, the prior floor and the snapshot quadrature, are estimated from stored histories rather than bounded ( $\bar{\zeta} = 0.035\text{--}0.039$ on the five Cellular Potts production replicates, the posterior estimate 0.2–0.5% in total variation from a large- $m$ reference), while the truncation constant and the rejection/redraw conditioning are bounded only. Since the factors multiply, the measured part is a partial accounting rather than a bound on the whole.                                                                                                                                                                                                                                                                                                                                                                                                                |
| <b>5(ii)</b> stabilization                                       | measured; <b>at the threshold</b>         | Not implied by the top- $k$ rule, and measured: on the five Cellular Potts production replicates the per-step drift decays as $\tau^{-0.7}$ to $\tau^{-0.95}$ (archive turnover 825–874 changes in about 12,900 draws, the $O(k \log n)$ record rate), while the distance to the limit decays as $\tau^{-b}$ with $b = 0.50\text{--}0.71$ over the full run and 0.25–0.72 away from the end (0.59 and 0.48 pooled), against the $b > \frac{1}{2}$ needed. A serial benchmark passes at $b \approx 0.6\text{--}0.76$ when the target is identified and fails at $b \approx 0.0\text{--}0.29$ when its parameters confound; the condition thus concerns the inference problem as much as the sampler. The central limit theorem is stated for the damped or frozen variants; consistency, needing no rate, is unaffected. The asynchronous replay of the proposal sequence enters here too, since a reporting rank’s arrival-ordered log need not reproduce the proposal each candidate came from. |
| <b>5</b> $\epsilon_\infty > 0$ (deterministic, for the CLT only) | <b>not satisfied</b>                      | A strictly positive <code>min.tol</code> forces $\epsilon_\infty \geq \text{min.tol} > 0$ , since $\{\epsilon_n\}$ is non-increasing and clipped from below, which is all Theorem 1 needs. Theorem 3 additionally needs $\epsilon_\infty$ deterministic, so that $f - \pi_{\epsilon_\infty}(f)$ is a fixed function; a floor alone does not give that, since a monotone data-driven schedule may settle above it at a run-dependent value. A schedule that reaches its floor in finite time does, and makes the $\epsilon$ terms of (A1) vanish identically. The runs reported here leave <code>min.tol</code> unset (zero is accepted), so neither part is <i>established</i> , though neither is disproved: a data-driven decreasing schedule may converge to a positive limit, even a deterministic one, but we do not verify it. Note also that the bandwidth a finite budget reaches is not itself an asymptotic $\epsilon_\infty$ .                                                        |
| <b>5</b> rate (A1)                                               | assumed                                   | Needed only in Step 1 and Step 2 of the proof of Theorem 3; consistency does not use it.                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |

**Table C2** A typical generation-staged ABC-SMC/PMC implementation (the pyABC baseline used here) versus the proposed generation-free method. The listed properties characterize pyABC as configured, not ABC-SMC as a family. The running sampler does carry the online snapshot buffer of §3.4; the posterior estimate rebuilds it from the history.

| Property           | Classical ABC-SMC                | This work                                 |
|--------------------|----------------------------------|-------------------------------------------|
| Update style       | generation, barrier-synchronized | event-driven (single-arrival), no barrier |
| Archive            | explicit, generational           | reconstructed, sliding top- $k$           |
| ABC likelihood     | hard threshold                   | smooth kernel                             |
| Importance weight  | against $q_{t-1}$ only           | balance heuristic over $\mathcal{S}$      |
| Archive state      | mutable, carried                 | none; reconstructed per call              |
| Posterior estimate | final population                 | pure function of the history              |

**Table D3** Resolved experiment configurations. Workers, per-run wall-clock budget, replicates (trials for SBC), archive size  $k$ , and methods (A: asynchronous; S: pyABC (matched kernel); R: rejection ABC; T: barrierized twin). The twin row covers the straggler, heterogeneity and Cellular Potts 50<sup>3</sup> studies at 16, 48 and 48–384 workers. <sup>†</sup> Simulation-limited: the budget is per-trial (SBC) or per-run (kill-and-resume) *simulations*, not wall-clock seconds; the twin, and the asynchronous re-run matched to it, stop at a fixed simulation count per level (§5.2, Table 2). <sup>‡</sup> Also run with AMIS on/off, a drain-after-deadline variant, and a steeper coupling regime (base delay 0.5 s, cap 8 s). <sup>§</sup> The Cellular Potts rows of these studies ran the benchmark’s earlier configuration (Appendix E).

| Experiment                             | Workers          | Budget (s)        | Reps | $k$      | Methods |
|----------------------------------------|------------------|-------------------|------|----------|---------|
| Gaussian mean                          | 48               | 300               | 5    | 100      | A,S,R   |
| g-and-k                                | 48               | 600               | 5    | 100      | A,S,R   |
| Lotka–Volterra                         | 48               | 600               | 5    | 100      | A,S     |
| Cellular Potts 50 <sup>3</sup>         | 48               | 3600              | 5    | 100      | A,S,R   |
| Cellular Potts 80 <sup>3</sup>         | 48               | 3600              | 3    | 100      | A,S,R   |
| Straggler                              | 16               | 300               | 5    | 100      | A,S     |
| Runtime heterogeneity                  | 48               | 60                | 5    | 100      | A,S     |
| Parameter-coupled runtime <sup>‡</sup> | 48               | 60                | 5    | 100      | A,S     |
| Strong scaling (LV)                    | 1–288            | 180               | 5    | 100/1000 | A,S     |
| Strong scaling (CPM) <sup>§</sup>      | 1–384            | 1800              | 3–5  | 100      | A,S     |
| Barrierized twin <sup>§</sup>          | 16 / 48 / 48–384 | sims <sup>†</sup> | 5    | 100      | T,A     |
| Sensitivity                            | 48               | 300               | 5    | 50/200   | A       |
| Ablation                               | 48               | 300               | 5    | 100      | A       |
| SBC (Gaussian, 1-D)                    | 48               | 20k <sup>†</sup>  | 1000 | 100      | A,S     |
| SBC (bimodal)                          | 48               | 20k <sup>†</sup>  | 1000 | 100      | A,S     |
| SBC (g-and-k, 4-D)                     | 48               | 40k <sup>†</sup>  | 1000 | 200      | A,S     |
| SBC (g-and-k, 2×)                      | 48               | 80k <sup>†</sup>  | 500  | 200      | A,S     |
| Kill-and-resume                        | 4                | 120k <sup>†</sup> | 1    | 100      | A       |

**Table F4** Barrierized twin under runtime heterogeneity ( $W = 48$ , lognormal runtime multiplier of scale  $\sigma$ , five replicates, medians of simulations per second of each sampler’s own elapsed wall clock). *Predicted* is from the injected law and the asynchronous run’s measured overhead. The  $\sigma = 0$  control costs almost nothing because these evaluations take 1 s and barrier latency is negligible against them; the cost is the wait for the slowest draw and grows with  $\sigma$ . pyABC degrades less than the twin because it discards latecomers instead of waiting for them. The lower block scores the posteriors against the analytic posterior at the bandwidth each schedule reached, then re-reported at the 100th order statistic of the same history (§6.4); no rank made  $k$  calls in the asynchronous run, so its schedule never ran at  $\sigma \leq 0.5$ , and the twin’s ran only at  $\sigma \leq 0.5$ .

| Heterogeneity $\sigma$                                                                                           | 0      | 0.5   | 1.0   | 1.5   | 2.0   |
|------------------------------------------------------------------------------------------------------------------|--------|-------|-------|-------|-------|
| Asynchronous, sims/s                                                                                             | 47.6   | 42.1  | 28.9  | 17.6  | 11.4  |
| Twin, barrier every $W$                                                                                          | 44.8   | 15.1  | 4.37  | 1.06  | 0.37  |
| Twin, barrier every 96                                                                                           | 45.1   | 19.8  | 6.50  | 1.75  | 0.45  |
| pyABC                                                                                                            | 23.1   | 18.9  | 9.91  | 4.35  | 2.39  |
| Ratio, measured (every $W$ )                                                                                     | 1.06×  | 2.8×  | 6.6×  | 16.6× | 29.6× |
| Ratio, predicted                                                                                                 | 1.00×  | 2.77× | 6.34× | 14.0× | 35.4× |
| Ratio, pyABC                                                                                                     | 2.1×   | 2.2×  | 2.9×  | 4.1×  | 4.8×  |
| <i>Posterior recovery, <math>W_1</math> to the analytic posterior (calls per rank: async 59–13, twin 440–10)</i> |        |       |       |       |       |
| Evaluations, async                                                                                               | 2832   | 2527  | 1731  | 1055  | 684   |
| Evaluations, twin                                                                                                | 21,120 | 6,912 | 2,112 | 672   | 480   |
| Bandwidth reached, async                                                                                         | 5      | 5     | 0.92  | 1.25  | 0.78  |
| Bandwidth reached, twin every $W$                                                                                | 0.36   | 2.4   | 5     | 5     | 5     |
| Async, at the schedule’s bandwidth                                                                               | 2.12   | 2.04  | 0.68  | 0.93  | 0.56  |
| Twin every $W$ , at the schedule’s bandwidth                                                                     | 0.19   | 1.43  | 2.20  | 2.22  | 2.28  |
| pyABC, last generation                                                                                           | 0.53   | 0.61  | 1.36  | 2.19  | 2.21  |
| Async, re-reported at $\epsilon_{(100)}$                                                                         | 0.011  | 0.011 | 0.010 | 0.013 | 0.042 |
| Twin every $W$ , re-reported at $\epsilon_{(100)}$                                                               | 0.015  | 0.013 | 0.012 | 0.040 | 0.203 |
| Async archive (top 100)                                                                                          | 0.018  | 0.017 | 0.016 | 0.014 | 0.019 |

**Table F5** The twin’s slowdown on Cellular Potts  $50^3$  in the benchmark’s earlier single-simulation configuration (Appendix E), factorized (five replicates, medians). Throughput is utilization over mean simulation time, so the ratio is the product of the two columns that follow. The utilization ratio is what the barrier costs; it agrees with the straggler factor measured inside the twin’s own generations and with the prediction from the asynchronous run’s runtime distribution. The duration ratio (the twin’s simulations running longer, with replicate means from 6 to 36 s against 5.1 s on every asynchronous replicate) is not a barrier effect and is not attributed here.

| Workers (= cores)                          | 48    | 96    | 192   | 384    |
|--------------------------------------------|-------|-------|-------|--------|
| Throughput ratio, async / twin             | 1.91× | 1.87× | 1.86× | 2.82×  |
| = utilization ratio                        | 1.18× | 1.19× | 1.21× | 1.21×  |
| × simulation-duration ratio                | 1.63× | 1.59× | 1.52× | 2.33×  |
| Straggler factor of the twin’s generations | 1.21  | 1.28  | 1.21  | 1.29   |
| Predicted from asynchronous timing         | 1.26× | 1.30× | 1.32× | (2.34× |

**Table F6** Strong scaling on Lotka–Volterra: median simulation throughput (sims/s) at a 180 s budget, over five replicates (Fig. 3, left, shows the inter-quartile spread). Asynchronous (ours) at both archive sizes vs. pyABC. Multi-node points use *fully packed* nodes (worker counts that are exact multiples of 48); at counts above 100 pyABC is given *population size equal to the worker count* so it can occupy every core (a population of 100 leaves the surplus idle), which is why it keeps scaling here. The asynchronous sampler peaks on one node and then declines as multi-node coordination dominates, so pyABC overtakes it beyond three nodes at  $k = 100$ . Enlarging the archive tenfold costs throughput at every scale, because the per-arrival proposal reconstruction is  $\mathcal{O}(k)$ , and moves that crossover in from beyond three nodes to three.

| Workers                     | 1   | 4   | 16   | 48   | 144  | 192  | 240  | 288  |
|-----------------------------|-----|-----|------|------|------|------|------|------|
| Nodes                       | <1  | <1  | <1   | 1    | 3    | 4    | 5    | 6    |
| Synchronous                 | 105 | 96  | 437  | 2952 | 3757 | 4322 | 4539 | 6034 |
| Asynchronous ( $k = 100$ )  | 149 | 369 | 1983 | 6240 | 4195 | 2615 | 2088 | 1729 |
| Asynchronous ( $k = 1000$ ) | 106 | 133 | 620  | 3804 | 2804 | 1957 | 1587 | 1336 |

**Table F7** Calibration sensitivity to the archive size  $k$  and the AMIS snapshot buffer  $S$  on the  $d$ -dimensional Gaussian-mean target (1000 trials per cell; entries are the mean signed deviation of empirical from nominal coverage over the 0.50/0.80/0.90/0.95 levels and the  $d$  parameters). Negative is over-confident, positive is conservative; 0.000 is exactly calibrated. **Left:** varying  $k$  at  $S = 20$ . **Right:** varying  $S$  at  $k = 100$ .

|          | archive size $k$ (at $S = 20$ ) |        |        |        |        |
|----------|---------------------------------|--------|--------|--------|--------|
|          | 50                              | 100    | 200    | 400    | 800    |
| $d = 1$  | -0.014                          | -0.003 | +0.006 | +0.006 | +0.074 |
| $d = 2$  | -0.020                          | -0.006 | +0.009 | +0.030 | +0.072 |
| $d = 4$  | +0.025                          | +0.064 | +0.094 | +0.133 | +0.063 |
| $d = 8$  | <b>-0.250</b>                   | +0.055 | +0.180 | +0.195 | +0.061 |
| $d = 16$ | -0.047                          | +0.028 | +0.049 | +0.053 | +0.055 |

|          | snapshot buffer $S$ (at $k = 100$ ) |        |        |        |
|----------|-------------------------------------|--------|--------|--------|
|          | 0                                   | 5      | 20     | 50     |
| $d = 1$  | <b>-0.088</b>                       | -0.001 | -0.003 | -0.005 |
| $d = 2$  | <b>-0.088</b>                       | -0.001 | -0.006 | -0.003 |
| $d = 4$  | +0.039                              | +0.068 | +0.064 | +0.064 |
| $d = 8$  | +0.065                              | +0.006 | +0.055 | +0.074 |
| $d = 16$ | -0.010                              | +0.010 | +0.028 | +0.034 |

**Table F8** The archive size on both of its axes at once. **Throughput:** median asynchronous throughput at the single-node peak (48 workers) on Lotka–Volterra, 180 s wall cap, five replicates ( $k \in \{100, 1000\}$  from the main scaling run,  $k \in \{50, 200, 400, 800\}$  from the matched  $k$ -frontier run), and its change relative to the  $k=100$  we use throughout. **Calibration:** worst absolute deviation of empirical from nominal coverage over the dimensions of the  $d$ -dimensional Gaussian-mean sweep at  $S=20$  (Table F7), with the dimension attaining it. **Crossover:** fully packed nodes at which pyABC (population = worker count) overtakes the asynchronous sampler. The sweep did not include  $k=1000$ .

| $k$  | sims/s at $W=48$ | vs. $k=100$ | worst $ \Delta\text{cov} $ | at    | pyABC overtakes |
|------|------------------|-------------|----------------------------|-------|-----------------|
| 50   | 6684             | +7%         | <b>0.250</b>               | $d=8$ | beyond 3 nodes  |
| 100  | 6240             | —           | <b>0.064</b>               | $d=4$ | beyond 3 nodes  |
| 200  | 5904             | −5%         | 0.180                      | $d=8$ | beyond 3 nodes  |
| 400  | 5393             | −14%        | 0.195                      | $d=8$ | 3 nodes         |
| 800  | 4331             | −31%        | 0.074                      | $d=1$ | 3 nodes         |
| 1000 | 3804             | −39%        | —                          | —     | 3 nodes         |